

Retrieval-Augmented Generation: Interview and Implementation Guide

This guide treats each component of a production RAG system as an individual interview topic. Code examples are deliberately small enough to run and modify; production systems additionally need authentication, observability, access control, and load testing.

Embeddings

1. Overview

An embedding maps an object such as text, an image, or a user into a fixed-length dense vector. In RAG, one encoder embeds document chunks offline and queries online so that semantically related items are close in the same vector space. Embeddings power retrieval, clustering, recommendations, duplicate detection, and anomaly detection.

2. Intuition

Think of an embedding space as a map: "car" and "automobile" receive nearby coordinates even though their characters differ, while "banana" lies farther away. A query is located on the same map, and nearby passages become evidence for the LLM.

3. Prerequisites

- Vectors, dot products, norms, cosine similarity, and matrix multiplication
- Tokenization and Transformer encoders
- Supervised learning, negatives, softmax, and cross-entropy
- Basic NumPy/PyTorch and batching

4. Core Concepts

Subtopic	Meaning and importance	Example	Common interview angle
Dense representation	A learned vector with mostly non-zero values	768 floats for a paragraph	Dense vs sparse vectors
Bi-encoder	Query and passage are encoded independently	$E_q(q)$, $E_d(d)$	Why it supports offline indexing
Similarity	A score expressing closeness	cosine or dot product	Effect of L2 normalization
Positive/negative pairs	Relevant and irrelevant query-document pairs	question and answer passage	Hard-negative mining

Subtopic	Meaning and importance	Example	Common interview angle
Pooling	Converts token states to one vector	CLS or mean pooling	Mean pooling vs CLS
Asymmetry	Queries and documents have different roles	short question vs long passage	Separate encoders/prompts

5. Algorithm / Working Process

1. Tokenize a query or chunk.
2. Run tokens through a Transformer encoder.
3. Pool token hidden states into one vector.
4. Optionally project and L2-normalize it.
5. Store document vectors during indexing.
6. At inference, embed the query and score it against stored vectors.

Training commonly samples a positive passage and several negatives per query. Backpropagation makes the positive score larger than negative scores. Inference requires only forward passes and nearest-neighbor search.

6. Mathematical Foundation

For encoder output token states $h_1, \dots, h_T$, masked mean pooling is

$$e = (\sum_i m_i h_i) / (\sum_i m_i).$$

Cosine similarity is

$$s(q, d) = (e_q^T e_d) / (||e_q||_2 ||e_d||_2).$$

If vectors are normalized, cosine similarity equals the dot product. A common in-batch contrastive loss is

$$L_i = -\log[\exp(s(q_i, d_i+)/\tau) / \sum_j \exp(s(q_i, d_j)/\tau)],$$

where τ is a temperature. Lower temperature sharpens score differences. Triplet loss is $\max(0, m + s(q, d-) - s(q, d+))$.

7. Practical Implementation

```
from sentence_transformers import SentenceTransformer
import numpy as np

texts = [
    "Reset a forgotten password from the account settings page.",
    "Paris is the capital of France.",
]

query = "How can I change my password?"

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
```

```
doc_vectors = model.encode(texts, normalize_embeddings=True)
query_vector = model.encode([query], normalize_embeddings=True)[0]

scores = doc_vectors @ query_vector # cosine because vectors are normalized
for i in np.argsort(scores)[::-1]:
    print(f"{scores[i]:.3f} {texts[i]}")
```

8. Code Explanation

`encode` turns texts into equal-dimensional vectors. Normalization makes their L2 norm one, so a fast matrix-vector product gives cosine scores. `argsort` ranks passages from most to least similar. In a real index, the document encoding happens once; only the query is encoded online.

9. Training / Evaluation

Prepare query-positive pairs from clicks, QA datasets, or synthetic questions. Split by document, user, and preferably time to prevent near-duplicate leakage. Evaluate retrieval with Recall@k, MRR, and nDCG; also inspect downstream answer accuracy. Important hyperparameters include encoder, dimension, maximum tokens, pooling, temperature, batch size, and negative strategy. Hard negatives usually help more than random negatives, but false negatives can damage training.

10. Complexity and Cost

Transformer encoding is roughly quadratic in token length because of self-attention and linear in batch size. Storing N float32 vectors of dimension d costs $4Nd$ bytes before index overhead. Quantization can reduce memory. Offline document embedding is normally the large one-time cost; query embedding adds online latency and often benefits from CPU batching or a small GPU.

11. Common Use Cases

- RAG passage retrieval and semantic FAQ matching
- Product or content recommendation
- Image-text search with multimodal encoders
- Clustering, deduplication, and topic discovery
- Classification using nearest examples

12. Common Mistakes

- Mixing vectors from different models or model versions
- Forgetting the model's required query/document prefixes
- Using dot product while assuming cosine normalization
- Truncating away the answer-bearing part of a chunk
- Evaluating only similarity examples instead of retrieval on the target corpus
- Randomly splitting near-duplicate documents and causing leakage

13. Edge Cases / Limitations

Embeddings can blur negation, exact numbers, rare identifiers, or subtle temporal differences. A vector compresses a long passage, so not every fact survives. Domain shift, multilingual imbalance, and adversarial

text reduce quality. Similarity means semantic relatedness, not necessarily entailment or factual correctness.

14. Variations

- **Sentence/document embeddings:** one vector per text; essential for placements and projects.
- **Token-level embeddings (ColBERT):** retain token vectors for late interaction; useful when higher accuracy justifies memory.
- **Sparse learned embeddings (SPLADE):** learned vocabulary weights; important in retrieval research.
- **Multimodal embeddings:** align images and text; useful for product and document search.
- **Matryoshka embeddings:** allow truncating dimensions; useful for latency-memory trade-offs.

15. Related Topics

Embeddings provide the representation; vector search provides the index. Dense retrieval uses learned embeddings, whereas BM25 uses lexical term statistics. Cross-encoders jointly read query and passage and are usually more accurate but cannot precompute independent passage vectors. Fine-tuning changes the embedding geometry; RAG uses that geometry to find evidence.

16. Interview Questions

1. **What is an embedding?** A learned fixed-length vector whose geometry encodes useful relationships between inputs.
2. **Why normalize embeddings?** It removes magnitude effects and makes dot product equal cosine similarity.
3. **Why is a bi-encoder fast?** Passage vectors are computed offline and reused for every query.
4. **Cosine vs Euclidean distance?** On unit vectors, squared Euclidean distance is $2 - 2\cos(q, d)$, so rankings are equivalent.
5. **What is mean pooling?** Averaging non-padding token states to make one sequence vector.
6. **What is a hard negative?** An irrelevant passage that looks plausible to the current retriever and gives a strong training signal.
7. **What is a false negative?** A passage treated as irrelevant even though it answers the query; it teaches the wrong geometry.
8. **Can embeddings prove an answer is true?** No. They estimate relatedness, not truth or entailment.
9. **Why can larger dimensions hurt?** They increase memory and search cost, and do not guarantee better task alignment.
10. **How would you detect embedding drift?** Version vectors, maintain a fixed retrieval set, and track Recall@k and score distributions during migration.

17. Practice Tasks

- Encode 100 sentences and implement cosine top-k with NumPy.
- Build semantic search over a public FAQ dataset.
- Compare normalized dot product, cosine, and Euclidean rankings.
- Debug a model-prefix mismatch that lowers Recall@10.
- Fine-tune a sentence encoder with hard negatives and measure improvement.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Support FAQ Matcher	Finds the closest verified answer	SentenceTransformers, FastAPI	CLINC150 or company FAQs	Shows serving and retrieval metrics
Multilingual Search	Retrieves across languages	multilingual-e5, FAISS	MIRACL	Demonstrates multilingual NLP
Image-Text Catalog Search	Searches products by description	CLIP, PyTorch	Fashion-MNIST plus captions or DeepFashion	Demonstrates multimodal embeddings

19. Quick Revision

- **Key idea:** encode meaning as geometry.
- **Main formula:** cosine similarity and contrastive softmax loss.
- **When to use:** reusable semantic comparison at scale.
- **Metrics:** Recall@k, MRR, nDCG, latency.
- **Common traps:** no normalization, wrong prefixes, weak negatives.
- **Interview one-liner:** a bi-encoder trades joint query-document interaction for precomputable, scalable vectors.

20. Final Cheat Sheet

Item	Summary
Definition	Learned dense representation of an input
Input/output	Text or image -> d -dimensional vector
Main steps	tokenize, encode, pool, normalize, compare
Key hyperparameters	model, dimension, max length, pooling, temperature
Metrics	Recall@k, MRR, nDCG, throughput
Pros / cons	Semantic and reusable / lossy, domain-sensitive
Best use cases	Semantic retrieval, clustering, recommendations

Vector Search

1. Overview

Vector search finds vectors nearest to a query vector under cosine similarity, dot product, or a distance metric. It turns embeddings into a scalable retrieval system. Exact search works for small collections; approximate nearest-neighbor (ANN) indexes such as HNSW, IVF, and product quantization trade a small amount of recall for major latency and memory gains.

2. Intuition

If each passage is a pin on a high-dimensional map, vector search asks which pins lie nearest the query pin. Checking every pin is exact but slow; ANN builds shortcuts or partitions so the search visits only promising regions.

3. Prerequisites

- Embeddings, norms, cosine, inner product, Euclidean distance
- Arrays, heaps, graphs, clustering, and basic complexity
- Recall@k, latency percentiles, and memory sizing

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
k-NN	Exact top-k over all vectors	matrix multiplication	Cost $O(Nd)$
ANN	Fast, possibly imperfect top-k	HNSW	Recall-latency trade-off
HNSW	Multi-layer navigable proximity graph	graph walk	M , $efConstruction$, $efSearch$
IVF	Search selected centroid lists	inverted cells	$nlist$ and $nprobe$
PQ	Compress subvectors into codebook IDs	byte codes	Memory vs distortion
Filtering	Restrict candidates by metadata	tenant or date	Pre- vs post-filtering

5. Algorithm / Working Process

Exact search scores the query against every vector, maintains top-k, and returns associated records. HNSW inserts each vector into proximity graphs; a query greedily traverses upper layers and explores a candidate set at the base layer. IVF trains centroids, assigns vectors to lists, probes the closest lists, and optionally decodes compressed vectors. Index building is offline; searching is online.

6. Mathematical Foundation

Nearest neighbors are $\arg \text{top-}k_d s(q, d)$. Exact dot-product search costs $O(Nd)$ per query. For normalized vectors, maximizing $q^T d$ equals minimizing Euclidean distance because $\|q - d\|^2 = 2 - 2q^T d$.

ANN quality is measured by $\text{Recall@}k = |\text{ANN}_k \cap \text{Exact}_k| / k$. System tuning optimizes a constrained objective such as maximizing recall subject to $p95_latency \leq L$ and memory $\leq M$.

7. Practical Implementation

```
import numpy as np

vectors = np.array([[1., 0.], [.8, .2], [0., 1.]], dtype="float32")
```

```
vectors /= np.linalg.norm(vectors, axis=1, keepdims=True)
query = np.array([.9, .1], dtype="float32")
query /= np.linalg.norm(query)

k = 2
scores = vectors @ query
candidate_ids = np.argpartition(scores, -k)[-k:]
ranked_ids = candidate_ids[np.argsort(scores[candidate_ids])[:-1]]
print([(int(i), float(scores[i])) for i in ranked_ids])
```

8. Code Explanation

Rows and query are unit-normalized. Matrix multiplication calculates all cosine scores. `argpartition` selects `k` candidates without fully sorting `N` values; only those `k` values are sorted. This remains exact scoring, not ANN, and is a strong baseline for small corpora.

9. Training / Evaluation

The search index itself is usually configured rather than supervised. Build a representative query set, compute exact neighbors as ground truth, then report ANN Recall@`k`, p50/p95/p99 latency, QPS, index build time, and memory. Tune HNSW `efSearch` or IVF `nprobe` on validation queries, never only on a few demos.

10. Complexity and Cost

- Exact: $O(Nd)$ query time and $O(Nd)$ storage.
- HNSW: empirical near-logarithmic search, but graph edges add memory; build is expensive.
- IVF: scans roughly selected lists; performance depends on distribution and `nprobe`.
- PQ: much lower storage and better cache behavior, with quantization error.
- Updates and deletes may require tombstones, graph maintenance, or rebuilds.

11. Common Use Cases

Semantic RAG, recommendation, image similarity, duplicate detection, nearest examples for few-shot prompts, and candidate generation for ranking systems.

12. Common Mistakes

- Comparing index recall against labels instead of exact-neighbor recall
- Choosing a distance inconsistent with model training
- Filtering after a tiny ANN result and returning too few items
- Ignoring embedding/index version compatibility
- Benchmarking average latency while tail latency fails
- Assuming a vector database automatically improves poor embeddings

13. Edge Cases / Limitations

ANN can miss the true nearest item, especially with aggressive compression or restrictive filtering. High-dimensional vectors may cluster poorly. Many small tenants can make global indexes inefficient. Frequent mutations, exact numeric lookup, and stringent access control require careful architecture.

14. Variations

- **Flat exact index:** best ground truth and small-corpus option; placement essential.
- **HNSW:** strong recall/latency and dynamic insertion; common in production.
- **IVF-Flat:** partitions then exactly scores candidates; useful at large scale.
- **IVF-PQ/PQ:** compresses vectors; useful when RAM dominates.
- **Disk ANN:** keeps much of index on SSD; research/large-scale relevance.

15. Related Topics

Embeddings define vector quality, while vector search approximates nearest-neighbor computation. Dense retrieval adds a task and corpus around vector search. Metadata filtering changes the candidate universe. Reranking corrects coarse first-stage order using a more accurate model.

16. Interview Questions

1. **Exact k-NN vs ANN?** Exact scores all vectors; ANN searches a subset and may lose recall.
2. **Why use HNSW?** It offers strong recall-latency trade-offs and supports inserts.
3. **What does efSearch do?** Controls HNSW candidate exploration; larger values improve recall and latency cost.
4. **What does nprobe do?** Controls how many IVF lists are searched.
5. **What is product quantization?** It quantizes vector subvectors with learned codebooks to reduce memory.
6. **How do you benchmark ANN?** Compare with exact top-k and measure recall, latency percentiles, QPS, and memory.
7. **Why can post-filtering fail?** Relevant filtered records may never enter the initial top-k.
8. **Can cosine use an inner-product index?** Yes, after L2 normalization.
9. **How do deletes work?** Often tombstones/lazy deletion followed by compaction or rebuild.
10. **When is brute force preferable?** Small or frequently changing collections, exactness needs, or a baseline.

17. Practice Tasks

- Implement exact top-k without sorting all scores.
- Compare exact search with an HNSW library on one million random vectors.
- Plot Recall@10 against p95 latency while varying search effort.
- Diagnose a filter that leaves fewer than k results.
- Quantize vectors and measure memory and recall loss.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
ANN Benchmark Lab	Compares Flat, HNSW, and IVF	FAISS, NumPy, Streamlit	SIFT1M	Demonstrates systems trade-offs
Similar-Image Finder	Retrieves visually similar images	CLIP, FAISS	CIFAR-10	End-to-end multimodal search

Project	What it does	Tech stack	Dataset	Resume value
Multi-tenant Vector API	Isolates tenant documents and filters	FastAPI, Qdrant/pgvector	Synthetic tenant corpus	Production architecture signal

19. Quick Revision

- **Key idea:** retrieve nearest vectors without scanning everything.
- **Main formula:** top-k similarity; ANN Recall@k against exact neighbors.
- **When to use:** large embedding collections.
- **Metrics:** recall, p95 latency, QPS, memory.
- **Common traps:** metric mismatch and filtering after too-small k.
- **Interview one-liner:** ANN exchanges a controlled amount of exact-neighbor recall for orders-of-magnitude better search efficiency.

20. Final Cheat Sheet

Item	Summary
Input/output	Query vector -> IDs, scores, metadata
Main steps	normalize, index, search candidates, top-k
Hyperparameters	k, metric, efSearch , nprobe , PQ size
Metrics	ANN Recall@k, p95 latency, QPS, RAM
Pros / cons	Scalable semantic lookup / approximation and index operations
Best use cases	Large static or moderately dynamic embedding corpora

Semantic Search

1. Overview

Semantic search retrieves results by intended meaning rather than only shared words. It normally embeds queries and documents, performs vector retrieval, and may rerank candidates. It is used for natural-language knowledge search, support, e-commerce, code search, and multilingual retrieval.

2. Intuition

A lexical engine may not connect "laptop will not power on" with "notebook startup failure." Semantic search places paraphrases close together, so meaning can match despite different words.

3. Prerequisites

- Text preprocessing and tokenization
- Embeddings, cosine similarity, and vector indexes
- Information-retrieval relevance judgments and ranking metrics

- Basic understanding of BM25 and reranking

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Intent	Information need behind query	"jaguar speed" is ambiguous	Query ambiguity
Semantic similarity	Meaning closeness in embedding space	paraphrases	Similarity vs relevance
Candidate retrieval	High-recall first stage	top 100 vectors	Recall requirement
Reranking	More precise ordering	cross-encoder top 20	Latency-quality trade-off
Relevance labels	Graded query-document judgments	0, 1, 2, 3	nDCG and annotation
Search UX	Snippets, facets, and clarification	highlighted answer	Offline vs online quality

5. Algorithm / Working Process

1. Parse and normalize documents while retaining useful metadata.
2. Create retrieval units and embeddings; build an ANN index.
3. Normalize or rewrite the incoming query when needed.
4. Retrieve high-recall candidates.
5. Apply permissions and metadata constraints safely.
6. Rerank and diversify candidates.
7. Return results/snippets or pass evidence to an LLM.

Training uses query-document relevance pairs; inference encodes one query and searches the precomputed corpus.

6. Mathematical Foundation

Semantic similarity often uses cosine $s(q,d)=q^Td/(||q|| ||d||)$. Retrieval training uses contrastive cross-entropy. Ranking quality uses

$DCG@k = \sum_{i=1}^k (2^{\{rel_i\}-1})/\log_2(i+1)$ and $nDCG@k = DCG@k / IDCG@k$.

MRR is $MRR = (1/|Q|) \sum_q 1/rank_q$, using the first relevant result. Semantic similarity is only a feature; relevance can also depend on freshness, authority, locale, and permissions.

7. Practical Implementation

```
from sentence_transformers import SentenceTransformer
import numpy as np
```

```

records = [
    {"title": "Battery troubleshooting", "text": "Steps when a notebook does not turn on."},
    {"title": "Fast animals", "text": "The cheetah is a very fast land animal."},
]
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
matrix = model.encode([r["title"] + ". " + r["text"] for r in records],
                      normalize_embeddings=True)

def search(query: str, k: int = 2):
    q = model.encode([query], normalize_embeddings=True)[0]
    ids = np.argsort(matrix @ q)[-k:][::-1]
    return [(records[i]["title"], float(matrix[i] @ q)) for i in ids]

print(search("My laptop has no power"))

```

8. Code Explanation

Title and body are embedded together because titles carry strong topic signals. The same encoder and normalization are used for query and documents. The function returns ranked titles and scores; a production engine would use ANN, filters, snippets, and a reranker.

9. Training / Evaluation

Build judgments from search logs plus human labels, separating train/test by time and query family. Report Recall@k for candidate generation, MRR/nDCG for ranking, zero-result rate, click-through or task success online, and latency. Guard against position bias in click labels. Evaluate head, tail, multilingual, typo, entity, negation, and exact-ID query slices.

10. Complexity and Cost

Costs include offline parsing/embedding/indexing, online query encoding, ANN search, and optional reranking. A bi-encoder search scales well; a cross-encoder costs approximately $O(KL^2)$ attention for K candidates of length L. Caching common queries can reduce latency but needs freshness and permission-aware keys.

11. Common Use Cases

Enterprise document search, customer support, product discovery, legal research, natural-language code search, help-center search, and the retrieval stage of RAG.

12. Common Mistakes

- Treating semantic similarity as complete business relevance
- Replacing a strong lexical baseline without measuring exact-keyword queries
- Training on biased clicks without debiasing
- Returning duplicate passages from the same document
- Ignoring locale, freshness, authority, or permissions

- Measuring only a few attractive examples

13. Edge Cases / Limitations

Dense semantic search often struggles with serial numbers, names, dates, negation, and very recent entities. Ambiguous queries may retrieve multiple intents. Relevant content missing from the corpus cannot be recovered. Similar but contradictory passages can rank highly.

14. Variations

- **Dense semantic search:** embedding similarity; core placement knowledge.
- **Hybrid search:** combines BM25 and dense ranks; often strongest production default.
- **Multilingual search:** shared cross-lingual space; useful for global products.
- **Personalized search:** adds user/context features; important when relevance varies by user.
- **Late-interaction search:** token-level matching; useful for research and high-quality systems.

15. Related Topics

Lexical search matches terms; semantic search matches learned meaning. Hybrid search combines their complementary signals. Vector search is the computational primitive, while semantic search is the end-user relevance system. Reranking improves precision after high-recall retrieval.

16. Interview Questions

1. **What makes search semantic?** Learned representations connect concepts and paraphrases beyond exact term overlap.
2. **Similarity vs relevance?** Similarity is one model score; relevance includes user intent and business/context constraints.
3. **Why keep BM25?** It excels at rare terms, identifiers, and exact matches.
4. **How evaluate candidate generation?** Recall@k against judged relevant documents.
5. **Why use nDCG?** It supports graded relevance and discounts lower ranks.
6. **What is query-document asymmetry?** Queries are short information needs; documents contain fuller answers and may need different prompts/encoders.
7. **How handle ambiguous queries?** Diversify results, use context, or ask a clarifying question.
8. **How handle new documents?** Incrementally parse/embed/index them and version the index.
9. **Why can click data mislead?** Ranking position and UI exposure influence clicks.
10. **When use a reranker?** When candidate recall is good but top-rank precision needs improvement.

17. Practice Tasks

- Compare TF-IDF and sentence embeddings on paraphrase queries.
- Create graded judgments for 50 queries and compute nDCG@10.
- Analyze failures by exact identifier, negation, and ambiguity slices.
- Add deduplication and result diversification.
- Run an online-style experiment with simulated clicks and position bias.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Natural-language Code Search	Finds functions from descriptions	CodeBERT, FAISS	CodeSearchNet	Strong NLP + software relevance
E-commerce Search	Combines meaning and filters	OpenSearch/FAISS, FastAPI	Amazon product data	Production search design
Multilingual Help Search	Searches English docs with other languages	multilingual-e5	MIRACL or translated FAQs	Global retrieval experience

19. Quick Revision

- **Key idea:** retrieve by intent and meaning, not only tokens.
- **Main formula:** embedding similarity; nDCG for ranked relevance.
- **Metrics:** Recall@k, MRR, nDCG, task success, latency.
- **Common traps:** ignoring exact terms and biased labels.
- **Interview one-liner:** semantic search is a relevance pipeline built around learned meaning, not merely a vector database query.

20. Final Cheat Sheet

Item	Summary
Input/output	Natural-language query -> ranked results
Main steps	encode, retrieve, filter, rerank, present
Hyperparameters	model, chunking, top-k, ANN effort, rerank depth
Metrics	Recall@k, MRR, nDCG, CTR/task success
Pros / cons	Handles paraphrases / weak on exact and ambiguous queries
Best use cases	Natural-language knowledge and catalog search

Chunking

1. Overview

Chunking divides documents into retrievable units that fit embedding and LLM context limits. Chunk boundaries determine whether a unit contains enough evidence to answer a question without introducing excessive unrelated text. It is one of the highest-impact RAG design decisions.

2. Intuition

Retrieving a whole textbook is too broad; retrieving one sentence may omit the definition's conditions. Good chunks resemble useful index cards: each is focused but self-contained.

3. Prerequisites

- Tokens, model context windows, document structures, and NLP sentence splitting
- Embeddings and retrieval metrics
- Basic parsing for Markdown, HTML, PDF, tables, and code

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Chunk size	Maximum tokens/characters per unit	300 tokens	Precision vs completeness
Overlap	Repeated boundary content	50 tokens	Recall vs duplication
Structural split	Honor headings/paragraphs	section-aware	Better semantics
Parent-child	Search small child, return larger parent	sentence -> section	Retrieval vs context granularity
Contextualization	Prefix chunk with title/path	policy > leave > eligibility	Lost-context problem
Special formats	Treat tables/code/conversations differently	whole table + caption	Parser-aware chunking

5. Algorithm / Working Process

1. Parse the source into typed blocks while preserving hierarchy and page/section IDs.
2. Remove boilerplate, repeated headers, and navigation.
3. Group coherent blocks up to a token budget.
4. Split oversized blocks at sentence or semantic boundaries.
5. Add limited overlap or neighboring references.
6. Attach title, section path, source, time, tenant, and ACL metadata.
7. Embed/index chunks and evaluate retrieval at multiple sizes.

6. Mathematical Foundation

For document token length T , chunk size c , and overlap $o < c$, the approximate number of fixed windows is

$$n = \text{ceil}(\max(0, T-c)/(c-o)) + 1.$$

More overlap increases storage and indexing approximately by factor $c/(c-o)$. Chunk selection balances answer containment probability against noise. One practical objective is $\text{Utility}(c) = \text{AnswerRecall}@k(c) - \lambda * \text{ContextTokens}(c)$ measured empirically.

7. Practical Implementation

```
import re

def chunk_text(text: str, max_words: int = 80, overlap: int = 15):
```

```

sentences = re.split(r"(?<=[.!?])\s+", text.strip())
chunks, current = [], []
for sentence in sentences:
    words = sentence.split()
    if current and len(current) + len(words) > max_words:
        chunks.append(" ".join(current))
        current = current[-overlap:] + words
    else:
        current.extend(words)
if current:
    chunks.append(" ".join(current))
return chunks

sample = "RAG retrieves evidence. Chunking controls retrieval granularity. Good
boundaries preserve meaning."
assert " ".join(chunk_text(sample, max_words=8, overlap=2)).count("RAG") == 1
print(chunk_text(sample, max_words=8, overlap=2))

```

8. Code Explanation

The regex preserves sentence boundaries better than blind character slicing. When adding a sentence would exceed the budget, the current chunk is emitted and its final words become overlap. Production code should use the target tokenizer and format-specific parsers; this example is intentionally dependency-light.

9. Training / Evaluation

Chunking is usually tuned, not trained. Create questions with answer-span and source labels. Sweep chunk size, overlap, and structural strategies; measure answer-span Recall@k, duplicate rate, context precision, downstream answer correctness, token cost, and latency. Split evaluation by document type because PDFs, tables, code, and prose behave differently.

10. Complexity and Cost

Parsing and chunk creation are $O(T)$. Smaller chunks increase count, embedding cost, index memory, duplicate candidates, and reranking load. Larger chunks consume context tokens and may dilute embeddings. Parent-child retrieval adds storage for both units but can reduce prompt fragmentation.

11. Common Use Cases

PDF QA, policies/manuals, codebase assistants, support knowledge bases, legal contracts, transcripts, and scientific-paper retrieval.

12. Common Mistakes

- Using characters while reasoning about model tokens
- Applying one chunk size to every format
- Splitting tables, headings, code blocks, or definitions from their qualifiers
- Adding large overlap that creates near-duplicates
- Losing source, page, hierarchy, or ACL metadata
- Tuning only downstream generation, making retrieval failures hard to see

13. Edge Cases / Limitations

Scanned PDFs may have broken reading order. Tables require headers with rows; code requires symbol boundaries; conversations require speaker/time grouping. A fact may depend on distant sections, so no local window is sufficient. Updates can shift offsets and complicate citations.

14. Variations

- **Fixed windows:** simple baseline; placement essential.
- **Recursive/structure-aware:** prefers headings, paragraphs, then sentences; strong project default.
- **Semantic chunking:** splits where embedding similarity changes; useful but more costly.
- **Parent-child:** retrieves fine units and supplies broader context; common advanced pattern.
- **Late chunking:** encodes long context before pooling chunks; research-relevant.

15. Related Topics

Chunking determines the document vectors used by dense retrieval and term statistics used by BM25. Retrieval selects chunks; context construction deduplicates and orders them. Multi-hop RAG may retrieve multiple chunks linked across documents. Metadata preservation enables filtering and citations.

16. Interview Questions

1. **Why chunk documents?** To create focused retrievable units within encoder and LLM limits.
2. **Small vs large chunks?** Small improves focus but loses context; large preserves context but adds noise and cost.
3. **Why overlap?** To avoid losing evidence across boundaries.
4. **Why not use huge overlap?** It expands the index and produces duplicate results/context.
5. **What is structure-aware chunking?** Splitting along headings, paragraphs, tables, or code symbols before size limits.
6. **What is parent-child retrieval?** Search small chunks but return a larger linked parent for generation.
7. **How choose chunk size?** Sweep on representative labeled questions and optimize retrieval plus answer metrics and cost.
8. **How chunk tables?** Preserve headers, caption, row groups, and table identity together.
9. **How chunk code?** Prefer functions/classes and attach file/module/symbol metadata.
10. **What metadata is essential?** Stable source ID, location, hierarchy, version, and permissions.

17. Practice Tasks

- Implement token-aware sliding windows.
- Compare fixed and heading-aware chunks on a documentation corpus.
- Measure duplicate top-k rate as overlap increases.
- Diagnose a PDF where the answer and heading are separated.
- Implement parent-child expansion and compare context precision.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Chunking Benchmark	Evaluates four strategies	Python, tiktoken, FAISS	SQuAD articles	Shows experimental rigor
PDF Structure RAG	Preserves pages/headings/tables	PyMuPDF, SentenceTransformers	Annual reports	Practical document AI
Code-Aware Chunker	Indexes symbols with hierarchy	tree-sitter, Qdrant	CodeSearchNet	Strong AI-engineering signal

19. Quick Revision

- **Key idea:** retrieval units must be focused and self-contained.
- **Main formula:** chunk count grows as stride `c-o` shrinks.
- **Metrics:** answer-span Recall@k, context precision, duplicates, tokens.
- **Common traps:** blind fixed splitting and excessive overlap.
- **Interview one-liner:** chunking is an information-boundary problem, not merely a token-limit operation.

20. Final Cheat Sheet

Item	Summary
Input/output	Parsed document -> chunks plus metadata
Main steps	parse, clean, group, split, overlap, annotate
Hyperparameters	chunk tokens, overlap, separators, parent size
Metrics	span Recall@k, duplicates, context precision, token cost
Pros / cons	Enables focused retrieval / can break context
Best use cases	Every document-based RAG pipeline

Retrieval

1. Overview

Retrieval selects a small evidence set from a large corpus for a query. In RAG it is the bridge between stored knowledge and generation. Good retrieval maximizes the chance that answer-bearing, authorized, current evidence reaches the generator while controlling latency and noise.

2. Intuition

A retriever acts like a librarian: it does not write the answer; it locates the few books and pages from which a careful writer can answer.

3. Prerequisites

- Chunking, inverted indexes, embeddings, and vector search
- Precision, recall, ranks, and relevance judgments
- Query processing, metadata, access control, and caching

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Candidate generation	Fast high-recall first stage	BM25/vector top 100	Recall before precision
Top-k	Number passed downstream	retrieve 50, rerank 10	Quality-cost tuning
Relevance	Evidence answers the information need	policy clause	Similarity is not enough
Corpus/index	Searchable chunks and metadata	versioned knowledge base	Freshness and deletes
Fusion	Combine multiple retrievers	RRF	Score calibration
Feedback	Use labels/logs to improve retrieval	judged queries	Leakage and bias

5. Algorithm / Working Process

1. Resolve user identity, tenant, locale, and conversation state.
2. Normalize, classify, or rewrite the query.
3. Apply mandatory access constraints.
4. Run one or more sparse/dense retrievers for high-recall candidates.
5. Merge, deduplicate, and optionally diversify results.
6. Rerank with a stronger model.
7. return evidence, scores, metadata, and trace information.

Training is optional for BM25 but central to dense retrievers. Inference must preserve document/embedding/index versions for reproducibility.

6. Mathematical Foundation

For relevant set R_q and returned top-k set A_q^k ,

$Recall@k = |R_q \cap A_q^k| / |R_q|$, and $Precision@k = |R_q \cap A_q^k| / k$.

MRR emphasizes the first relevant hit. Candidate retrieval normally prioritizes Recall@k because a reranker or generator cannot recover evidence that was never retrieved. Reciprocal rank fusion uses $RRF(d) = \sum_r \frac{1}{(c + rank_r(d))}$.

7. Practical Implementation

```
from collections import defaultdict

def reciprocal_rank_fusion(rankings, c=60, k=5):
    scores = defaultdict(float)
    for ranked_ids in rankings:
        for rank, doc_id in enumerate(ranked_ids, start=1):
            scores[doc_id] += 1.0 / (c + rank)
    return sorted(scores, key=scores.get, reverse=True)[:k]

bm25_ids = ["d3", "d1", "d8", "d2"]
dense_ids = ["d2", "d3", "d9", "d1"]
assert reciprocal_rank_fusion([bm25_ids, dense_ids], k=1) == ["d3"]
print(reciprocal_rank_fusion([bm25_ids, dense_ids]))
```

8. Code Explanation

Each retriever contributes a rank-based score. RRF avoids assuming that BM25 and cosine scores share a scale. Documents strong in multiple rankings accumulate more evidence. The constant `c` limits how strongly the very first ranks dominate.

9. Training / Evaluation

Create query-document relevance labels and freeze a test set. Evaluate Recall@k, MRR, nDCG, coverage, duplicate rate, and retrieval latency. Slice by query class, source, recency, and permission. For trained retrievers, sample hard negatives from strong baselines, monitor false negatives, and validate downstream grounded-answer accuracy.

10. Complexity and Cost

Cost depends on corpus size, index type, number of retrieval routes, top-k, filters, and reranking depth. Sparse and ANN retrieval are usually milliseconds to tens of milliseconds; encoder and reranker calls add compute. Larger k improves recall only until noise and downstream cost dominate. Cache keys must include user-visible corpus/version and authorization scope.

11. Common Use Cases

Open-domain QA, enterprise knowledge assistants, legal discovery, support automation, recommendation candidate generation, code assistants, and multimodal search.

12. Common Mistakes

- Optimizing only generator prompts when evidence is absent
- Using Precision@k alone for candidate retrieval
- Fusing raw scores from incomparable systems
- Retrieving before enforcing tenant/ACL constraints
- Keeping stale or duplicate chunks after document updates

- Evaluating with generated answers but no retrieval labels

13. Edge Cases / Limitations

No retriever can return absent or unindexed knowledge. Conflicting, time-sensitive, or permission-dependent evidence needs explicit handling. Vague queries may require clarification. Long-tail entity and multi-hop questions may need query expansion or iterative retrieval.

14. Variations

- **Sparse retrieval:** lexical, explainable, excellent for exact terms.
- **Dense retrieval:** semantic bi-encoder, strong for paraphrases.
- **Hybrid retrieval:** fuses both; common production choice.
- **Multi-stage retrieval:** cheap candidates then expensive reranking.
- **Adaptive retrieval:** skips or expands retrieval based on query need; advanced project/research topic.

15. Related Topics

Chunking defines retrieval units. BM25 and dense encoders are alternative candidate generators. Vector search executes dense nearest-neighbor lookup. Reranking optimizes ordering; context construction decides what finally enters the prompt. Retrieval evaluation must be separated from generation evaluation.

16. Interview Questions

1. **What is retrieval's role in RAG?** Select evidence from an external corpus before generation.
2. **Why prioritize Recall@k first?** Missing evidence cannot be restored downstream.
3. **What controls top-k?** Recall gains versus reranking, prompt token, and latency costs.
4. **What is multi-stage retrieval?** A fast broad retriever followed by a slower precise ranker.
5. **Why use RRF?** It combines rankings without calibrating incompatible raw scores.
6. **How test retrieval independently?** Use query-document relevance labels and ranking metrics.
7. **How handle permissions?** Enforce filters in candidate generation and verify them before context use.
8. **What are hard negatives?** Plausible non-relevant results used to sharpen training.
9. **How handle stale documents?** Version ingestion, delete/tombstone old chunks, and test freshness.
10. **How diagnose wrong answers?** Check corpus presence, chunking, candidate recall, reranking, context, then generation in that order.

17. Practice Tasks

- Implement Recall@k and MRR from relevance labels.
- Fuse BM25 and dense rankings with RRF.
- Build a 100-query golden set and categorize misses.
- Debug a stale-index and duplicate-chunk incident.
- Add an adaptive top-k policy and compare cost/quality.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Retrieval Evaluation Harness	Benchmarks sparse, dense, hybrid	Python, BEIR, FAISS	BEIR	Strong measurable IR skills
Permission-Aware Docs Search	Retrieves only authorized chunks	FastAPI, PostgreSQL/pgvector	Synthetic enterprise docs	Security-aware RAG design
Adaptive Retriever	Chooses route and k by query	PyTorch, BM25, vector DB	Natural Questions subset	Advanced cost-quality work

19. Quick Revision

- **Key idea:** place sufficient, relevant evidence in the candidate set.
- **Main formulas:** Recall@k, MRR, nDCG, RRF.
- **When to use:** any large external knowledge source.
- **Common traps:** low recall, unsafe filters, stale data, uncalibrated fusion.
- **Interview one-liner:** retrieval is the recall-oriented evidence stage; generation quality is upper-bounded by what it returns.

20. Final Cheat Sheet

Item	Summary
Input/output	Query + scope -> ranked evidence candidates
Main steps	process, filter, retrieve, fuse, dedupe, rerank
Hyperparameters	top-k, retriever weights/routes, filters, rerank depth
Metrics	Recall@k, MRR, nDCG, latency, coverage
Pros / cons	Fresh auditable evidence / index and relevance failures
Best use cases	RAG, QA, search, recommendation candidates

Reranking

1. Overview

Reranking reorders a small candidate set produced by a fast retriever using a more accurate relevance model or richer features. It improves top-result precision without applying an expensive model to the entire corpus. RAG systems use it to choose the few chunks that deserve scarce context-window space.

2. Intuition

The first retriever creates a shortlist of applicants from thousands of resumes; the reranker reads each shortlisted resume carefully and orders the best matches.

3. Prerequisites

- Candidate retrieval, top-k, embeddings, and BM25
- Transformer encoders, logits, softmax, and supervised classification
- MRR, nDCG, pairwise/listwise ranking, and latency measurement

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
First-stage recall	Candidate set must contain relevant items	retrieve 100	Reranker cannot recover misses
Relevance scoring	Query-aware score per candidate	cross-encoder logit	Score is not necessarily probability
Ranking depth	Number of candidates reranked	top 50	Quality-latency balance
Pointwise/pairwise/listwise	Different ranking objectives	relevant label vs preference	Loss choice
Diversification	Avoid redundant top evidence	MMR	Relevance vs coverage
Calibration	Interpret scores or thresholds	abstain below threshold	Dataset shift

5. Algorithm / Working Process

1. Retrieve K candidates using BM25, dense search, or hybrid fusion.
2. Construct a query-candidate input for each result.
3. Score candidates independently or jointly with a reranking model.
4. Combine relevance with optional freshness, authority, or diversity rules.
5. Sort and retain the best $k \ll K$ chunks.
6. Pass scores and provenance into context construction.

Training uses relevance labels or preferences. Inference batches candidate pairs to keep accelerators utilized.

6. Mathematical Foundation

Pointwise binary relevance uses $L = -[y \log \sigma(s) + (1-y) \log(1-\sigma(s))]$. Pairwise ranking uses

$$L_{\text{pair}} = -\log \sigma(s(q, d_+) - s(q, d_-)).$$

Maximum marginal relevance diversifies selection:

$$\text{MMR}(d) = \lambda \text{Rel}(q, d) - (1 - \lambda) \max_{d' \in S} \text{Sim}(d, d').$$

The first term rewards relevance; the second penalizes redundancy with already-selected set S .

7. Practical Implementation

```
from sentence_transformers import CrossEncoder

query = "What is the leave carry-forward policy?"
candidates = [
    "Employees may carry forward up to five unused annual-leave days.",
    "Leave requests must be entered in the HR portal.",
    "The office is closed on public holidays.",
]

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
scores = reranker.predict([(query, text) for text in candidates])
ranked = sorted(zip(candidates, scores), key=lambda x: x[1], reverse=True)
for text, score in ranked:
    print(round(float(score), 3), text)
```

8. Code Explanation

Each query-passage pair is jointly tokenized, allowing token-level interaction before one relevance score is emitted. Sorting by the raw model score is enough for ranking; applying sigmoid does not change order. Batching is essential when reranking many candidates.

9. Training / Evaluation

Train with judged positives and challenging negatives from the deployed first-stage retriever. Split by query/document family to avoid leakage. Measure nDCG@k, MRR, Precision@k, downstream context precision, answer accuracy, and added p95 latency. Tune candidate depth and model size. Re-evaluate when the first-stage retriever changes because its candidate distribution changes.

10. Complexity and Cost

For K candidates and pair length L , a Transformer cross-encoder costs roughly $O(KL^2)$ attention and cannot precompute candidate scores. GPU batching, smaller models, truncation, and reranking only uncertain queries control cost. API rerankers add per-document cost and network latency.

11. Common Use Cases

RAG evidence selection, web search, product ranking, recommendation refinement, passage ranking, duplicate-aware result selection, and selecting few-shot examples.

12. Common Mistakes

- Reranking too few candidates when first-stage order is weak
- Reranking hundreds of long chunks without batching
- Training only on easy random negatives
- Treating raw logits as calibrated confidence

- Optimizing nDCG while allowing unauthorized content
- Returning many near-identical chunks from one source

13. Edge Cases / Limitations

The reranker is bounded by candidate recall. Long passages are truncated; relevance may depend on metadata not included in the model input. Domain shift and ambiguous queries lower reliability. A relevance model may prefer a topical passage that is obsolete or contradicted by a more authoritative source.

14. Variations

- **Cross-encoder:** highest common accuracy, higher cost; essential for interviews.
- **Bi-encoder second stage:** cheaper but weaker interaction; useful for large candidate sets.
- **Late interaction:** token-level MaxSim such as ColBERT; important in advanced retrieval.
- **LLM reranking:** listwise or pairwise prompting; useful for complex relevance but costly.
- **Rule/feature learning-to-rank:** mixes authority, freshness, clicks, and text scores; production search relevance.

15. Related Topics

Retrieval maximizes candidate recall; reranking improves top-rank precision. Cross-encoders are the standard neural reranker. Context construction uses reranked outputs but may still deduplicate, diversify, or enforce token budgets. Hybrid search can improve the pool before reranking.

16. Interview Questions

1. **Why rerank?** Fast retrievers use coarse representations; reranking applies deeper query-document interaction to a small set.
2. **Can reranking fix low Recall@K?** No; missing documents are unavailable to it.
3. **Why are cross-encoders accurate?** Query and candidate tokens attend to one another jointly.
4. **Why are they expensive?** Every query-candidate pair requires a Transformer forward pass.
5. **How choose rerank depth?** Sweep K against nDCG/answer quality and tail latency.
6. **What negatives should training use?** Hard negatives from the same first-stage system, with false-negative checks.
7. **Pointwise vs pairwise?** Pointwise predicts labels independently; pairwise directly learns relative preference.
8. **What is MMR?** A greedy method balancing query relevance and redundancy.
9. **Should scores be thresholded?** Only after calibration on representative data; ranking logits alone are not confidence.
10. **How prove reranking helps RAG?** Show retrieval ranking gains and downstream grounded-answer gains at acceptable latency.

17. Practice Tasks

- Rerank BM25 top-50 with a CrossEncoder.
- Plot nDCG@10 and latency as rerank depth changes.
- Implement pairwise loss from positive/negative scores.
- Diagnose truncation on long policy chunks.

- Add MMR after reranking and measure duplicate reduction.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Two-stage Passage Ranker	BM25 candidates plus cross-encoder	Pyserini, HF	MS MARCO	Classic IR system
Cost-Aware Reranker	Routes only uncertain queries	PyTorch, FastAPI	BEIR subset	Shows latency-quality optimization
Evidence Diversifier	Reranks and removes redundancy	SentenceTransformers, MMR	Multi-document QA	Strong RAG context design

19. Quick Revision

- **Key idea:** expensive precision over a cheap high-recall shortlist.
- **Main formulas:** pairwise logistic loss and MMR.
- **Metrics:** nDCG, MRR, Precision@k, p95 latency.
- **Common traps:** low candidate recall, long inputs, easy negatives.
- **Interview one-liner:** retrieve broadly, rerank deeply, and send only the best non-redundant evidence to the LLM.

20. Final Cheat Sheet

Item	Summary
Input/output	Query + K candidates -> reordered candidates/scores
Main steps	pair construction, scoring, sorting, diversification
Hyperparameters	candidate K, final k, max length, model, batch size
Metrics	nDCG@k, MRR, Precision@k, latency
Pros / cons	Better top precision / extra compute, bounded by recall
Best use cases	RAG and search with valuable top positions

Context Construction

1. Overview

Context construction converts retrieved records into a bounded, ordered, attributable prompt payload. It is not simple concatenation: the system must enforce permissions, remove duplicates, preserve citations, balance coverage, handle conflicts, and keep the most useful evidence within the model's token budget.

2. Intuition

Retrieval brings a pile of research notes. Context construction edits those notes into a clean briefing: labeled sources, no repeats, clear ordering, and only material relevant to the question.

3. Prerequisites

- Retrieval scores, chunk metadata, reranking, tokenization
- Prompt roles, context windows, and instruction hierarchy
- Deduplication, access control, and citation/provenance concepts

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Token budget	Maximum evidence tokens available	4,000 of 8,000 tokens	Budget allocation
Provenance	Stable source label and location	[S2, p.7]	Citation traceability
Deduplication	Remove overlapping chunks	same paragraph twice	Wasted tokens
Ordering	Arrange evidence for model use	strongest first	Lost-in-the-middle effect
Compression	Extract relevant sentences/summaries	contextual compression	Information loss
Conflict handling	Preserve competing claims and authority	old vs new policy	Do not silently merge

5. Algorithm / Working Process

1. Re-check authorization and discard stale/invalid records.
2. Canonicalize IDs and deduplicate exact or highly overlapping chunks.
3. Group nearby chunks or expand parents when continuity matters.
4. Detect conflicts, source authority, and timestamps.
5. Select a relevance-diverse subset under a token budget.
6. Format each item with an unambiguous source ID and metadata.
7. Add explicit instructions to use only evidence, cite claims, and abstain if insufficient.
8. Reserve tokens for the question, system instructions, and answer.

6. Mathematical Foundation

Selection resembles a knapsack problem. With chunk utility u_i , token cost t_i , and budget B :

maximize $\sum_i u_i x_i$, subject to $\sum_i t_i x_i \leq B$, $x_i \in \{0,1\}$.

A redundancy-aware utility can be $u_i = \text{relevance}_i - \lambda \max_{j \in S} \text{similarity}(i,j)$.

Context precision is the fraction of supplied claims/chunks actually useful; context recall measures whether required evidence is present.

7. Practical Implementation

```
def build_context(chunks, token_budget=120):
    """Greedy budgeter; chunks are already permission-checked and reranked."""
    selected, seen, used = [], set(), 0
    for chunk in chunks:
        normalized = " ".join(chunk["text"].lower().split())
        if normalized in seen or used + chunk["tokens"] > token_budget:
            continue
        seen.add(normalized)
        used += chunk["tokens"]
        selected.append(
            f'[{chunk["id"]}] {chunk["title"]}\n{chunk["text"]}'
        )
    return "\n\n".join(selected)

items = [
    {"id": "S1", "title": "Leave Policy", "text": "Five days may carry forward.",
     "tokens": 8},
    {"id": "S1-copy", "title": "Leave Policy", "text": "Five days may carry
forward.", "tokens": 8},
]
context = build_context(items)
assert context.count("Five days") == 1
print(context)
```

8. Code Explanation

The function assumes security filtering already happened, then deduplicates normalized exact text and greedily respects a token budget. Stable labels are rendered next to content for citations. Production implementations use the actual model tokenizer, near-duplicate detection, source-grouping, and explicit conflict metadata.

9. Training / Evaluation

No training is required for rule-based construction, but learned compressors or selectors need query-evidence labels. Measure context precision/recall, answer correctness, citation entailment, unsupported-claim rate, prompt tokens, and latency. Perform ablations on order, budget, number of sources, parent expansion, and compression. Use adversarial documents containing prompt injections.

10. Complexity and Cost

Exact deduplication is linear with hashing. Pairwise semantic deduplication is $O(K^2d)$ for K candidates, usually acceptable after retrieval. Compression adds model calls and risks losing qualifiers. More context increases input-token cost and attention work, and can reduce quality through distraction despite fitting the nominal window.

11. Common Use Cases

Document QA, cited research summaries, policy assistants, multi-document comparison, support bots, code assistants, and evidence packets for agents.

12. Common Mistakes

- Concatenating top-k without deduplication or budgeting
- Trusting retrieved text as instructions instead of untrusted data
- Omitting source IDs or losing chunk-to-document mapping
- Filling the entire context window and leaving little answer budget
- Summarizing evidence without retaining exact supporting text
- Silently mixing contradictory or differently dated sources

13. Edge Cases / Limitations

The best answer may require more evidence than the budget permits. Tables, code, and images need specialized serialization. Source text can contain malicious prompt injection. Very similar chunks may have one crucial different qualifier, so aggressive deduplication can be wrong. Ordering effects vary by model.

14. Variations

- **Top-k concatenation:** baseline; useful only for simple systems.
- **MMR/diverse selection:** reduces repeats; high practical value.
- **Contextual compression:** extracts query-relevant spans; useful with long chunks.
- **Parent expansion:** restores surrounding context; useful for definitions and code.
- **Hierarchical context:** summaries plus exact passages; important for long-document and multi-hop RAG.

15. Related Topics

Reranking estimates individual relevance; context construction optimizes the evidence set as a whole. Chunking defines available granularity. Generation follows the prompt but needs citation labels and abstention rules. Hallucination reduction depends on context quality and instruction/data separation.

16. Interview Questions

1. **Why not concatenate top-k?** Results may duplicate, conflict, exceed budget, or waste tokens on low-value text.
2. **What is context precision?** The proportion of supplied context that is relevant/useful for the answer.
3. **What is lost in the middle?** Models can underuse evidence placed away from the beginning/end of long context.
4. **How allocate tokens?** Reserve system/query/answer space, then select evidence by utility per token and coverage.
5. **How preserve citations?** Assign stable source IDs and retain exact mappings to document locations.
6. **How handle conflicts?** Include source dates/authority, surface disagreement, and avoid unsupported reconciliation.
7. **What is contextual compression?** Extracting or summarizing only query-relevant content before prompting.
8. **How defend against prompt injection?** Treat context as untrusted quoted data, isolate instructions, filter, and constrain tools/actions.
9. **Why diversify?** Multiple independent facts/sources may be more useful than repetitive top results.

10. **How evaluate construction?** Context recall/precision, citations, answer quality, tokens, and adversarial safety tests.

17. Practice Tasks

- Build a token-aware greedy context packer.
- Implement exact and cosine near-duplicate removal.
- Compare strongest-first, strongest-last, and interleaved ordering.
- Debug a prompt-injection passage that overrides instructions.
- Implement parent expansion under a fixed token budget.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Evidence Packer	Optimizes relevance, diversity, and tokens	Python, tiktoken	HotpotQA contexts	Shows algorithmic prompt design
Citation-Safe QA	Maps every sentence to exact passages	FastAPI, vector DB, LLM	Natural Questions	Demonstrates auditability
Injection-Resistant RAG	Tests malicious retrieved documents	Python, prompt/eval harness	Custom adversarial corpus	Security-focused AI engineering

19. Quick Revision

- **Key idea:** turn retrieved candidates into a safe, diverse evidence brief.
- **Main formula:** budgeted utility/knapsack with redundancy penalty.
- **Metrics:** context precision/recall, tokens, citation entailment.
- **Common traps:** concatenation, duplicate evidence, injection, no provenance.
- **Interview one-liner:** context construction is set selection and safe serialization under a token budget.

20. Final Cheat Sheet

Item	Summary
Input/output	Ranked chunks -> formatted bounded evidence context
Main steps	authorize, dedupe, expand/compress, budget, label, order
Hyperparameters	evidence budget, max sources, diversity, compression
Metrics	context precision/recall, tokens, citation accuracy
Pros / cons	Focus and traceability / compression and ordering risk
Best use cases	Every production RAG prompt

Generation

1. Overview

Generation is the stage where an autoregressive language model produces an answer conditioned on the user query, instructions, conversation state, and retrieved evidence. Its job in RAG is synthesis: combine supported facts, communicate uncertainty, and attach verifiable citations without inventing missing information.

2. Intuition

The generator is a writer handed a research packet. A good writer answers the exact question, cites the packet, and says when the packet is insufficient rather than filling gaps from imagination.

3. Prerequisites

- Transformer decoders, tokenization, attention, and next-token prediction
- Conditional probability, softmax, cross-entropy, and decoding
- Prompt roles, context construction, citations, and safety constraints

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Conditioning	Generate from query and evidence	prompt contains [S1]	RAG vs closed-book generation
Decoding	Choose tokens from predicted distribution	greedy/top-p	Determinism vs diversity
Grounding instruction	Restrict factual claims to context	"If absent, say unknown"	Helps but does not guarantee
Citation generation	Link answer claims to sources	[S1]	Citation correctness vs presence
Abstention	Decline unsupported answers	insufficient evidence	Confidence and thresholds
Structured output	Constrain schema	JSON answer/sources	Validation and retries

5. Algorithm / Working Process

1. Serialize trusted system instructions separately from untrusted evidence.
2. Insert the user question and labeled context within the model's window.
3. At each step, compute logits for the next token.
4. Apply decoding constraints/temperature and select a token.
5. Continue until an end token or length limit.
6. Parse and validate structure/citations.
7. Optionally verify claims against cited passages and retry or abstain.

Base training minimizes next-token loss; instruction/factual behavior may be improved by SFT or preference tuning. RAG itself usually requires no generator weight update.

6. Mathematical Foundation

For prompt $x=(\text{instructions}, q, D)$ and answer tokens y ,

$$P(y|x)=\prod_{t=1}^T P(y_t \mid x, y_{<t}).$$

Training minimizes $L=-\sum_t \log P_{\theta}(y_t|x,y_{<t})$. Temperature modifies logits:

$p_i=\text{softmax}(z_i/T)$; $T<1$ makes output sharper. Top-p sampling retains the smallest token set whose cumulative probability exceeds p . Factual correctness is not optimized directly by next-token likelihood.

7. Practical Implementation

```
from transformers import pipeline

generator = pipeline("text2text-generation", model="google/flan-t5-small")
context = "[S1] Employees may carry forward at most five annual-leave days."
prompt = f"""Answer only from the evidence. Cite the source ID.
If the evidence is insufficient, say: Insufficient evidence.

Evidence:
{context}

Question: How many leave days can be carried forward?
Answer: """"

result = generator(prompt, max_new_tokens=64, do_sample=False)[0]
["generated_text"]
print(result)
```

8. Code Explanation

The prompt defines evidence use, citation expectation, and abstention behavior. Greedy decoding (`do_sample=False`) reduces unnecessary randomness for factual QA. The small model is for practice; production requires model selection, safety controls, structured validation, and claim-level evaluation.

9. Training / Evaluation

Generator fine-tuning is optional; if used, prepare query-context-answer examples including unanswerable cases and correct citations. Split by source documents and time. Evaluate answer correctness, faithfulness/groundedness, completeness, citation precision/recall, refusal accuracy, format validity, latency, and tokens. Tune model, temperature, output limit, prompt, and evidence budget on a validation set.

10. Complexity and Cost

Prefill processes the whole prompt and grows roughly quadratically with sequence length under standard attention; KV caching makes each decode step reuse previous keys/values but memory grows with layers, length, and batch. Output latency is sequential and measured by time-to-first-token and tokens/second. Long evidence and verbose answers dominate cost.

11. Common Use Cases

Cited question answering, summarization across documents, support replies, report drafting, structured extraction with evidence, code explanation, and research assistants.

12. Common Mistakes

- Assuming retrieved context forces factual output
- Using high temperature for deterministic enterprise QA
- Asking for citations without validating that they entail claims
- Failing to include unanswerable examples in evaluation
- Mixing trusted instructions and untrusted retrieved content
- Letting output format errors reach downstream systems unchecked

13. Edge Cases / Limitations

Models may ignore evidence, follow malicious text inside it, cite irrelevant sources, merge contradictory claims, or rely on parametric memory. Long contexts suffer attention dilution. Numeric reasoning, tables, and exact quotations may need tools or extractive methods. Deterministic decoding is repeatable, not automatically correct.

14. Variations

- **Extractive QA:** selects source spans; useful when faithfulness dominates fluency.
- **Abstractive generation:** synthesizes prose; standard RAG.
- **Constrained/structured generation:** grammar or JSON schema; important in AI engineering.
- **Citation-aware generation:** trains/prompts claim-source alignment; important for research assistants.
- **Tool-augmented generation:** uses calculators/code/search for operations the model should not approximate.

15. Related Topics

Context construction supplies evidence and labels. Hallucination reduction combines retrieval, prompting, validation, and abstention. Fine-tuning changes behavior but does not replace fresh retrieval. Decoding controls variability; it cannot compensate for missing evidence.

16. Interview Questions

1. **What does the generator do in RAG?** Produces an answer conditional on the query and retrieved evidence.
2. **Does greedy decoding guarantee truth?** No; it chooses high-probability tokens, not verified facts.
3. **What does temperature do?** Rescales logits, controlling distribution sharpness and randomness.
4. **Why use low temperature for QA?** It reduces variation where creative diversity is unnecessary.
5. **What is grounding?** Making factual claims supported by supplied evidence.
6. **What is abstention?** Explicitly declining when evidence is insufficient or unreliable.
7. **How validate citations?** Check cited source existence and whether each passage entails its claim.
8. **Why can long context hurt?** Noise, attention dilution, positional effects, latency, and cost.
9. **RAG vs fine-tuning for knowledge?** RAG supplies updateable evidence at inference; fine-tuning changes weights/behavior.

10. **How measure generation separately?** Give gold evidence and test correctness/faithfulness, isolating the retriever.

17. Practice Tasks

- Compare greedy, temperature, and top-p decoding on factual questions.
- Enforce a JSON answer/citation schema and validate it.
- Create answerable and unanswerable cases and measure refusal accuracy.
- Diagnose a correct answer with a wrong citation.
- Add a calculator tool for numerical evidence questions.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Cited Policy Assistant	Answers and cites exact clauses	HF/LLM API, FastAPI	Public policy manuals	Practical grounded QA
Structured Evidence Extractor	Returns JSON fields plus citations	Pydantic, constrained decoding	SEC filings	Reliable AI integration
Answer Verifier Pipeline	Generates then checks claims	NLI model, LLM, vector DB	FEVER	Shows factuality engineering

19. Quick Revision

- **Key idea:** synthesize an answer conditionally from evidence.
- **Main formula:** autoregressive conditional likelihood.
- **Metrics:** correctness, groundedness, citation quality, refusal accuracy.
- **Common traps:** confusing probability with truth and citation presence with support.
- **Interview one-liner:** the generator is a conditional synthesizer; retrieval supplies evidence, while validation enforces trust.

20. Final Cheat Sheet

Item	Summary
Input/output	Instructions + query + evidence -> answer/citations
Main steps	prompt, prefill, decode, parse, validate
Hyperparameters	model, temperature, top-p, max tokens, schema
Metrics	correctness, faithfulness, citation precision, latency
Pros / cons	Fluent synthesis / can hallucinate or miscite
Best use cases	Evidence-grounded natural-language and structured answers

Hallucination Reduction

1. Overview

Hallucination reduction is the layered engineering of fewer unsupported, incorrect, or fabricated model claims. RAG helps by supplying evidence, but reliable systems also need corpus quality, strong retrieval, context controls, calibrated abstention, citation verification, and monitoring. Reduction is realistic; complete elimination is not.

2. Intuition

Giving a student an open book lowers guessing, but the student can still open the wrong page, misunderstand it, or invent a citation. Reliability requires the right book, instructions to use it, and checking the submitted answer.

3. Prerequisites

- Retrieval and generation failure modes
- Probability calibration, precision/recall, and NLI/entailment
- Prompt injection, provenance, and evaluation-set design

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Intrinsic hallucination	Contradicts supplied evidence	says 10 when source says 5	Faithfulness
Extrinsic hallucination	Adds unsupported information	invents an exception	Completeness vs support
Retrieval grounding	Supply relevant authoritative evidence	current policy	RAG is necessary, not sufficient
Abstention	Refuse when support is weak	"Insufficient evidence"	Precision-coverage trade-off
Verification	Check claims/citations after generation	NLI or LLM judge	Judge reliability
Provenance	Trace claims to exact sources	sentence-level citations	Auditability

5. Algorithm / Working Process

1. Curate authoritative, versioned documents and remove stale duplicates.
2. Retrieve with high recall, safe filters, and a reranker.
3. Construct concise context with explicit source labels and conflict metadata.
4. Instruct the generator to use evidence, cite claims, and abstain.
5. Generate at low randomness for factual tasks.

6. Decompose the answer into claims and verify each against cited text.
7. Reject, revise, or abstain on unsupported claims.
8. Log failure category and feed it into evaluation and corpus/retrieval fixes.

6. Mathematical Foundation

Claim support rate can be defined as

$\text{Faithfulness} = \text{supported_answer_claims} / \text{verifiable_answer_claims}.$

Citation precision is $\text{supported_citations} / \text{all_citations}$; citation recall is $\text{claims_with_supporting_citation} / \text{claims_requiring_citation}$. Abstention creates a selective prediction trade-off: coverage is the fraction answered, while selective risk is error rate among answered items. Choose threshold t to minimize risk subject to required coverage or cost.

7. Practical Implementation

```
def supported_by_context(answer: str, required_facts: list[str], context: str) -> bool:
    """Tiny deterministic guard for exact critical facts; use NLI for paraphrases."""
    normalized_context = " ".join(context.lower().split())
    return all(fact.lower() in normalized_context for fact in required_facts)

context = "Employees may carry forward five annual-leave days."
answer = "Employees can carry forward five days. [S1]"

if not supported_by_context(answer, ["five"], context) or "[S1]" not in answer:
    answer = "Insufficient verified evidence."
assert answer.startswith("Employees")
print(answer)
```

8. Code Explanation

The check illustrates a post-generation gate for a critical exact fact and required citation. It is intentionally narrow: substring checks do not establish semantic entailment. Real verification extracts atomic claims, retrieves cited spans, applies NLI/rules/tools, and records uncertainty rather than trusting a single judge blindly.

9. Training / Evaluation

Create answerable, unanswerable, conflicting-source, stale-source, numeric, and adversarial-injection test cases. Human experts label claim correctness and support. Report faithfulness, answer correctness, citation precision/recall, abstention precision/recall, coverage-risk curves, and severity-weighted errors. Evaluate retriever-oracle and gold-context conditions to localize failures.

10. Complexity and Cost

Each defense adds latency: reranking, multiple retrieval attempts, claim decomposition, NLI/LLM verification, or regeneration. The cheapest controls are better corpus hygiene, clear prompts, low temperature, and deterministic validations. Use heavier verification for high-risk claims rather than every conversational sentence.

11. Common Use Cases

Healthcare information support, legal/policy assistants, financial research, enterprise knowledge bots, customer support, cited summaries, and any workflow where a plausible wrong answer has material cost.

12. Common Mistakes

- Claiming RAG eliminates hallucination
- Using self-reported model confidence as calibrated probability
- Counting any citation as valid support
- Evaluating factuality only with another unvalidated LLM
- Forcing an answer even when no evidence exists
- Fixing prompts when the real failure is stale or missing retrieval data

13. Edge Cases / Limitations

Sources themselves can be wrong or contradictory. Entailment models fail on numbers, tables, and domain language. A verifier can share the generator's biases. Correct world knowledge absent from context may be labeled unsupported by design. Prompt injection can manipulate generation or verification if evidence is treated as instructions.

14. Variations

- **Grounded prompting:** simplest layer; required but insufficient.
- **Self-consistency/self-check:** generate/check multiple times; useful but correlated errors remain.
- **NLI verification:** tests claim entailment; strong for textual claims.
- **Corrective RAG:** evaluates retrieval and re-retrieves or searches externally; advanced projects.
- **Selective generation:** answer only above a support threshold; essential in high-risk settings.

15. Related Topics

Retrieval quality and context construction are upstream controls. Generation and decoding affect unsupported elaboration. Evaluation defines hallucination precisely and measures it. Fine-tuning can teach abstention/citation style but cannot guarantee current facts. Tool use is preferable for arithmetic and database-grounded values.

16. Interview Questions

1. **What is hallucination in RAG?** A claim unsupported by or contradictory to valid evidence/ground truth.
2. **Does RAG eliminate it?** No; retrieval, interpretation, and generation can all fail.
3. **Intrinsic vs extrinsic?** Intrinsic contradicts context; extrinsic adds claims not supported by it.
4. **How does abstention help?** It trades answer coverage for lower error among answered questions.
5. **Are citations enough?** No; cited passages must actually entail the associated claims.

6. **How localize a hallucination?** Test corpus presence, retrieval, context, and generation using gold intermediate inputs.
7. **Can temperature zero still hallucinate?** Yes; deterministic high-probability text can be wrong.
8. **How handle conflicting sources?** Preserve conflict, rank authority/recency, and state uncertainty.
9. **What is selective risk?** Error rate on the subset the system chooses to answer.
10. **Best high-risk architecture?** Authoritative data, access-safe retrieval, constrained outputs, tools for exact operations, verification, and human escalation.

17. Practice Tasks

- Label claims in 50 RAG answers as supported/unsupported/contradicted.
- Build an answerability classifier and plot risk versus coverage.
- Create adversarial prompt-injection documents.
- Debug whether ten failures originate in retrieval or generation.
- Add claim-level citation verification with an NLI model.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
RAG Fact Checker	Extracts and verifies answer claims	HF NLI, FastAPI	FEVER	Demonstrates reliability engineering
Abstaining Medical FAQ	Answers only with supported guidance	vector DB, calibrated classifier	MedQuAD	Shows high-stakes caution
Hallucination Test Bench	Injects missing/conflicting/malicious evidence	Python, pytest, LLM judge + humans	Custom suite	Strong AI evaluation portfolio

19. Quick Revision

- **Key idea:** reliability is defense in depth, not one prompt.
- **Main formulas:** claim support, citation precision/recall, risk-coverage.
- **Metrics:** faithfulness, correctness, citations, abstention quality.
- **Common traps:** fake citations and uncalibrated self-confidence.
- **Interview one-liner:** RAG reduces knowledge hallucination only when retrieval is relevant and the generated claims are verified against evidence.

20. Final Cheat Sheet

Item	Summary
Input/output	Query/evidence/draft -> supported answer or abstention
Main steps	ground, constrain, cite, verify, revise/abstain
Hyperparameters	support threshold, retrieval k, verifier, temperature

Item	Summary
Metrics	faithfulness, correctness, citation precision, risk-coverage
Pros / cons	Safer and auditable / added cost, verifier errors
Best use cases	Factual and high-consequence RAG

Evaluation

1. Overview

RAG evaluation measures the pipeline at multiple levels: corpus/ingestion, retrieval, context, generation, safety, and system performance. A single end-to-end score hides root causes. Strong evaluation uses a versioned representative dataset, deterministic metrics where possible, expert or rubric-based judgments where necessary, slices, and regression gates.

2. Intuition

Evaluating only the final answer is like grading a delivery company without checking whether failures came from the warehouse, route, driver, or damaged package. Component metrics show where to fix the system.

3. Prerequisites

- Train/validation/test design, sampling, confidence intervals, and data leakage
- Precision, recall, ranking metrics, classification metrics
- Retrieval, grounded generation, citations, and latency/cost measurement

4. Core Concepts

Layer	What to measure	Example	Interview angle
Corpus	coverage, freshness, parse quality	answer source exists	Upper bound
Retrieval	Recall@k, MRR, nDCG	gold chunk in top 10	Candidate vs rank quality
Context	precision, recall, redundancy	useful evidence ratio	Token efficiency
Generation	correctness, faithfulness, completeness	supported answer	Separate correctness/support
Citation	precision, recall, entailment	claim cites correct span	Presence is insufficient
System	p95 latency, cost, availability, safety	tokens/query	Quality-cost frontier

5. Algorithm / Working Process

1. Define task, users, risk, and acceptable behavior including abstention.
2. Sample representative real queries and important edge/adversarial cases.
3. Record gold answers, relevant documents/chunks, evidence spans, and rubrics.

4. Freeze a test set; use a separate development set for tuning.
5. Run component and end-to-end evaluations with versioned configs.
6. Review disagreement and a random sample manually.
7. Slice results by query/source/language/risk class.
8. Compare quality, latency, and cost; gate regressions before deployment.
9. Validate offline improvements with controlled online metrics.

6. Mathematical Foundation

Retrieval uses Recall@k , $\text{MRR} = (1/|Q|) \sum 1/\text{rank}_q$, and nDCG. Answer exact match is $1[\text{prediction}=\text{gold}]$; token F1 uses overlap precision/recall. For a binary success rate $\hat{p}$, an approximate 95% confidence interval is $\hat{p} \pm 1.96 \sqrt{\hat{p}(1-\hat{p})/n}$; bootstrap intervals work for complex metrics. Paired evaluation should compare per-query differences, not unrelated aggregate estimates.

7. Practical Implementation

```
def recall_at_k(retrieved, relevant, k):
    relevant = set(relevant)
    return len(set(retrieved[:k]) & relevant) / max(1, len(relevant))

def reciprocal_rank(retrieved, relevant):
    relevant = set(relevant)
    return next((1 / rank for rank, doc in enumerate(retrieved, 1)
                 if doc in relevant), 0.0)

cases = [
    ({'retrieved': ['d2', 'd1'], 'relevant': ['d1']}),
    ({'retrieved': ['d3', 'd4'], 'relevant': ['d5']}),
]
print("Recall@2:", sum(recall_at_k(c["retrieved"], c["relevant"], 2) for c in
cases) / len(cases))
print("MRR:", sum(reciprocal_rank(c["retrieved"], c["relevant"]) for c in cases) /
len(cases))
```

8. Code Explanation

`recall_at_k` checks how much labeled relevant evidence appears within the cutoff. Reciprocal rank rewards placing the first relevant result high. Averaging per query prevents queries with many labels from dominating. Production evaluation should also retain per-case outputs for slicing and error analysis.

9. Training / Evaluation

Dataset construction is the central task. Use real traffic with privacy controls plus targeted cases; split by time/document to prevent leakage. Have multiple annotators and clear relevance/support rubrics. Track inter-annotator agreement and adjudicate high-risk disagreement. Hyperparameters are tuned only on development data. Online A/B metrics include resolution rate, search reformulation, user correction, escalation, latency, and cost—not clicks alone.

10. Complexity and Cost

Deterministic retrieval metrics are cheap once labels exist; expert annotation is expensive. LLM judges scale but introduce bias, order sensitivity, and model-version drift, so calibrate against humans. Full factorial sweeps can be expensive; use staged experiments and cache invariant embeddings/retrieval outputs. Report cost per successful answer, not only cost per request.

11. Common Use Cases

Model/index selection, chunking experiments, prompt regression, vendor comparison, deployment gates, incident diagnosis, safety testing, and cost-quality optimization.

12. Common Mistakes

- Using only answer similarity such as BLEU/ROUGE for open-ended QA
- Tuning on the test set or leaking source duplicates across splits
- Evaluating only answerable easy questions
- Trusting an LLM judge without human calibration
- Reporting averages without slices or confidence intervals
- Changing retriever, prompt, and model together and losing attribution

13. Edge Cases / Limitations

Many questions have multiple valid answers and evidence sets. Gold labels are incomplete, making good novel retrieval look wrong. Reference answers become stale. Human experts disagree. Offline metrics may not predict user success because presentation and workflow matter. Safety events are rare but high severity, so average scores underweight them.

14. Variations

- **Reference-based evaluation:** compare with gold answers; essential for placements.
- **Reference-free/rubric evaluation:** judge correctness/support from evidence; useful for open-ended output.
- **LLM-as-judge:** scalable qualitative scoring; important with calibration caveats.
- **Online A/B evaluation:** measures real task impact; production requirement.
- **Adversarial/red-team evaluation:** tests injection, leakage, conflict, and unsafe advice; essential for high-risk systems.

15. Related Topics

Retrieval metrics diagnose candidate quality; context metrics measure what the generator sees. Hallucination evaluation separates correctness from faithfulness. Ranking uses nDCG/MRR, while generation requires claim- and citation-level rubrics. MLOps supplies dataset/model versioning, traces, dashboards, and regression gates.

16. Interview Questions

1. **How evaluate RAG?** Separately score corpus, retrieval, context, answer, citations, safety, latency, and cost.
2. **Why is Recall@k important?** It measures whether necessary evidence reaches downstream stages.

3. **MRR vs nDCG?** MRR focuses on first relevant result; nDCG supports multiple graded relevant results.
4. **Correctness vs faithfulness?** Correctness compares to truth; faithfulness checks support by supplied context.
5. **How evaluate citations?** Check source validity and claim-level entailment for precision/recall.
6. **Why include unanswerable questions?** To measure abstention and prevent forced hallucinations.
7. **How avoid leakage?** Split by source/time/entity and freeze the test set.
8. **Are LLM judges reliable?** Useful but biased and version-dependent; calibrate with humans and deterministic checks.
9. **How compare two systems statistically?** Use paired per-query differences with bootstrap or an appropriate paired test.
10. **What should block deployment?** Regression beyond defined thresholds in critical quality, safety, authorization, or latency slices.

17. Practice Tasks

- Implement Recall@k, MRR, DCG, and nDCG.
- Create a 100-case RAG set with source spans and unanswerables.
- Compare two chunk sizes with paired bootstrap confidence intervals.
- Audit an LLM judge against two human annotators.
- Build a failure dashboard by retrieval/generation/safety category.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
RAG Eval Harness	Versioned component/end-to-end tests	Python, Pandas, MLflow	HotpotQA/BEIR	Excellent AI-engineering evidence
Judge Calibration Study	Compares LLM and human rubrics	Python, statsmodels	Custom 300 answers	Research-quality analysis
Quality-Cost Dashboard	Plots Pareto frontier across configs	Streamlit, Pandas	Experiment logs	Shows product/system judgment

19. Quick Revision

- **Key idea:** measure each pipeline boundary and the full user outcome.
- **Main formulas:** Recall@k, MRR, nDCG, confidence intervals.
- **Metrics:** correctness, faithfulness, citations, safety, p95 latency, cost.
- **Common traps:** leakage, incomplete labels, one aggregate score.
- **Interview one-liner:** a useful RAG evaluation tells you both whether the system failed and which stage caused it.

20. Final Cheat Sheet

Item	Summary
Input/output	Versioned cases + traces -> metrics, slices, failures
Main steps	define, sample, label, run, slice, compare, gate
Hyperparameters	k, judge rubric, thresholds, bootstrap samples
Metrics	retrieval, context, answer, citation, safety, cost/latency
Pros / cons	Diagnosable progress / labels and judges are costly
Best use cases	Development, regression testing, deployment decisions

Hybrid Search

1. Overview

Hybrid search combines sparse lexical retrieval, usually BM25, with dense semantic retrieval. Lexical search captures exact terms, names, numbers, and rare identifiers; dense retrieval captures paraphrases and conceptual similarity. Fusion gives robust candidate recall across mixed query types and is a common production RAG default.

2. Intuition

One detective searches for exact fingerprints; another follows semantic clues. Combining their shortlists catches cases either detective would miss alone.

3. Prerequisites

- Inverted indexes, TF-IDF/BM25, embeddings, and ANN
- Rank metrics, score normalization, and top-k selection
- Metadata filters, deduplication, and reranking

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Sparse branch	Term-based candidates	error code E1042	Exact-match strength
Dense branch	Meaning-based candidates	"cannot sign in"	Paraphrase strength
Early/late fusion	Combine features before/after retrieval	weighted scores/RRF	When scores are comparable
RRF	Rank-only fusion	sum reciprocal ranks	Robust baseline
Candidate budget	Per-branch depth	50 sparse + 50 dense	Recall and cost

Subtopic	Meaning and importance	Example	Interview angle
Reranking	Jointly score fused candidates	cross-encoder top 100	Final precision

5. Algorithm / Working Process

1. Parse the query and enforce shared metadata/permission scope.
2. Run BM25 and dense ANN retrieval, often concurrently.
3. Normalize scores or convert each list to ranks.
4. Merge by document/chunk ID and calculate a fusion score.
5. Optionally add exact-match boosts or business constraints.
6. Rerank the fused candidate set and select context.

No training is required for RRF; weighted fusion and learned rankers are tuned on validation judgments.

6. Mathematical Foundation

Weighted score fusion is $S(d) = \alpha * S_{\text{dense}}'(d) + (1 - \alpha) * S_{\text{sparse}}'(d)$, where prime denotes comparable normalization. Min-max and z-score normalization can be unstable across query distributions.

RRF avoids raw scores: $RRF(d) = \sum_{r \text{ in retrievers}} w_r / (c + \text{rank}_r(d))$. A document absent from one list contributes zero for that branch. Tune α , branch depths, or weights using Recall@k/nDCG, not intuition alone.

7. Practical Implementation

```
from collections import defaultdict

def rrf(named_rankings, weights=None, c=60):
    weights = weights or {name: 1.0 for name in named_rankings}
    fused = defaultdict(float)
    for name, ids in named_rankings.items():
        for rank, doc_id in enumerate(ids, 1):
            fused[doc_id] += weights[name] / (c + rank)
    return sorted(fused.items(), key=lambda item: item[1], reverse=True)

rankings = {
    "bm25": ["error-E1042", "login-guide", "network-guide"],
    "dense": ["login-guide", "password-reset", "error-E1042"],
}
print(rrf(rankings)[:3])
```

8. Code Explanation

Each branch contributes according to rank rather than incompatible raw score. A result appearing high in both lists usually wins, while unique candidates remain available. Stable record IDs are necessary for deduplication. A production pipeline reranks the merged set and applies security constraints inside both branches.

9. Training / Evaluation

Build query slices for exact IDs, entities, paraphrases, natural-language questions, and mixed queries. Compare BM25, dense, fused, and fused-plus-reranker using Recall@k, MRR/nDCG, zero-result rate, latency, and cost. Tune branch depth, fusion constant/weights, and reranking depth on validation. A learned fusion model needs unbiased relevance labels and feature monitoring.

10. Complexity and Cost

Hybrid search pays for two indexes and two query paths. Run branches in parallel so latency approaches the slower branch plus fusion. Memory includes both postings and vectors. Fusion over $K_s + K_d$ candidates is cheap; reranking the larger union is the main additional cost. Caching must respect index versions and filters.

11. Common Use Cases

Enterprise search, technical support with error codes, product search with model numbers and descriptions, legal search, biomedical literature, code/documentation assistants, and general-purpose RAG.

12. Common Mistakes

- Adding raw BM25 and cosine scores without calibration
- Using different ACL filters in sparse and dense branches
- Retrieving too few results from one branch before fusion
- Assuming hybrid always wins every query slice
- Failing to deduplicate the union
- Paying for two paths but evaluating only end-to-end anecdotes

13. Edge Cases / Limitations

Both branches fail when the corpus lacks the answer or parsing is broken. Dense and sparse rankings can reinforce the same wrong topic. Very restrictive filters can starve ANN retrieval. Hybrid doubles operational surfaces and may not justify its cost for a narrow, exact-only corpus.

14. Variations

- **RRF**: robust rank fusion with little tuning; placement/project default.
- **Weighted normalized scores**: useful when scores are calibrated and labels exist.
- **Learned-to-rank fusion**: combines text, dense, authority, freshness; production relevance.
- **Sparse learned retrieval + dense**: SPLADE with embeddings; research-relevant.
- **Query routing**: select sparse, dense, or both by query type; useful for cost optimization.

15. Related Topics

BM25 supplies sparse rankings; dense retrieval supplies semantic rankings. RRF is a retrieval fusion method, not a reranker that jointly reads text. A cross-encoder improves final order after fusion. Metadata filtering must be consistently applied to both indexes.

16. Interview Questions

1. **Why hybrid search?** Sparse and dense retrieval have complementary exact-match and semantic strengths.
2. **What is RRF?** A rank-based sum of reciprocal rank contributions from multiple retrievers.
3. **Why not add raw scores?** BM25 and cosine have different, query-dependent scales.
4. **How choose branch top-k?** Sweep candidate recall and reranking cost on representative queries.
5. **Does hybrid require training?** RRF does not; weights or learned fusion can use labels.
6. **How handle duplicates?** Merge on stable chunk/document IDs before reranking.
7. **How preserve security?** Apply equivalent pre-filters/ACL constraints in every branch and recheck outputs.
8. **What query benefits most from BM25?** Rare identifiers, proper nouns, quoted phrases, and exact numbers.
9. **What query benefits most from dense?** Paraphrases, concepts, and vocabulary mismatch.
10. **How show hybrid value?** Per-query paired gains and slice metrics over both single retrievers, including latency/cost.

17. Practice Tasks

- Implement RRF and weighted score fusion.
- Compare BM25, dense, and hybrid on BEIR.
- Create exact-ID and paraphrase query slices.
- Debug inconsistent tenant filters across branches.
- Build a lightweight query router and test whether it saves latency.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Hybrid Technical Support	Matches error codes and natural language	OpenSearch, SentenceTransformers	Stack Exchange data	Realistic retrieval mix
Fusion Benchmark	Compares RRF, weighted, learned fusion	Pyserini, FAISS, LightGBM	BEIR	Strong IR experimentation
Routed Search API	Selects sparse/dense/both	FastAPI, classifier, vector DB	Custom query taxonomy	Cost-aware architecture

19. Quick Revision

- **Key idea:** fuse exact lexical and semantic evidence.
- **Main formula:** weighted fusion or RRF.
- **Metrics:** Recall@k, nDCG, latency, per-query win rate.
- **Common traps:** raw-score addition and inconsistent filters.
- **Interview one-liner:** hybrid search improves robustness by letting BM25 recover exact signals and dense retrieval recover meaning.

20. Final Cheat Sheet

Item	Summary
Input/output	Query -> fused sparse+dense candidates
Main steps	retrieve branches, normalize/rank, merge, rerank
Hyperparameters	per-branch k, alpha/RRF c, weights, rerank depth
Metrics	Recall@k, MRR/nDCG, latency, cost
Pros / cons	Robust across query types / two indexes and paths
Best use cases	Mixed natural-language and exact-term corpora

BM25

1. Overview

BM25 is a probabilistic lexical ranking function built on term frequency, inverse document frequency, and document-length normalization. It is training-free, fast with inverted indexes, interpretable, and exceptionally strong for exact terms, rare entities, codes, and domain vocabulary. It remains a mandatory RAG baseline.

2. Intuition

A document is relevant when it contains important query words. Repeating a word helps but eventually adds little; matching a rare word matters more than matching "the"; unusually long documents should not win merely because they contain more words.

3. Prerequisites

- Tokenization, term/document frequency, logarithms, and inverted indexes
- Precision, recall, ranking, and basic probability intuition
- Stopwords, stemming/lemmatization, fielded documents

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
TF saturation	Repetition has diminishing returns	3 occurrences not 3x value	Parameter k1
IDF	Rare terms carry more information	product code	Negative/variant IDF formulas
Length normalization	Correct long-document term opportunities	manual vs paragraph	Parameter b
Inverted index	Term -> postings with frequencies	refund -> [(d1,2)]	Efficient query time

Subtopic	Meaning and importance	Example	Interview angle
Analyzer	Tokenization/normalization pipeline	lowercase/stem	Query-index consistency
Field boosts	Title/body treated differently	title match stronger	BM25F

5. Algorithm / Working Process

1. Analyze every document into terms.
2. Build postings with document IDs, term frequencies, lengths, and corpus statistics.
3. Analyze a query with the compatible analyzer.
4. Fetch postings only for query terms.
5. Compute each term's saturated TF, IDF, and length-normalized contribution.
6. Sum contributions and return top-k via a heap.

BM25 has no gradient training; k_1 , b , analyzers, and field weights are tuned using validation relevance labels.

6. Mathematical Foundation

A common form is

$$\text{BM25}(q,d) = \sum_{t \in q} \text{IDF}(t) * \frac{[f(t,d)(k_1+1)]}{[f(t,d)+k_1(1-b+b|d|/\text{avgdl})]}.$$

One stable IDF is $\log(1 + (N-\text{df}(t)+0.5)/(\text{df}(t)+0.5))$. k_1 controls TF saturation (often around 1.2-2.0). b controls length normalization: $b=0$ disables it; $b=1$ fully normalizes by relative document length.

7. Practical Implementation

```
import math
from collections import Counter

docs = ["reset password account", "password policy minimum length", "annual leave policy"]
tokens = [d.lower().split() for d in docs]
avgdl, n = sum(map(len, tokens)) / len(tokens), len(tokens)

def bm25(query, k1=1.5, b=0.75):
    dfs = Counter(term for doc in tokens for term in set(doc))
    scores = []
    for doc in tokens:
        tf, score = Counter(doc), 0.0
        for term in query.lower().split():
            idf = math.log(1 + (n - dfs[term] + 0.5) / (dfs[term] + 0.5))
            f = tf[term]
            score += idf * f * (k1 + 1) / (f + k1 * (1 - b + b * len(doc) / avgdl))
        scores.append(score)
    return sorted(enumerate(scores), key=lambda x: x[1], reverse=True)
```

```
print(bm25("password policy"))
```

8. Code Explanation

Document frequencies count each term at most once per document. At query time, every document receives the sum of query-term contributions. The numerator rewards frequency while the denominator saturates it and normalizes length. This teaching implementation scans all documents; a real inverted index evaluates only postings for query terms.

9. Training / Evaluation

There is no model training. Tune analyzer, chunking, **k1**, **b**, field weights, phrase boosts, and top-k on a validation set. Evaluate Recall@k, MRR, nDCG, zero-result rate, latency, and exact-entity slices. Avoid leakage by separating document versions and query templates across test boundaries.

10. Complexity and Cost

Indexing is linear in total terms. Storage is proportional to postings, compressed efficiently in mature engines. Query time depends primarily on postings lengths for query terms, not all **N** documents; top-k algorithms can skip low-impact postings. BM25 runs well on CPUs and is much cheaper than neural encoding/reranking.

11. Common Use Cases

Web/document search, log and error-code search, legal discovery, product catalogs, source-code search, enterprise RAG candidate retrieval, and hybrid search.

12. Common Mistakes

- Calling BM25 a vector semantic model
- Using incompatible analyzers for documents and queries
- Removing meaningful stopwords in phrases or negation
- Ignoring field structure and document length distribution
- Comparing BM25 on whole documents with dense search on chunks
- Dismissing BM25 before testing rare-term/entity slices

13. Edge Cases / Limitations

BM25 cannot directly match unseen synonyms or paraphrases with no term overlap. Tokenization is difficult for code, compound words, and languages without spaces. Term independence ignores word order unless phrase/proximity features are added. Very short chunks can make length normalization less informative.

14. Variations

- **BM25+**: adds a lower-bounded TF contribution; useful for long-document bias.
- **BM25L**: alternative length correction.
- **BM25F**: combines fields with weights; important for product/document search.
- **Query expansion**: adds synonyms or generated terms; useful for vocabulary mismatch.

- **Learned sparse retrieval:** SPLADE learns term weights; research-relevant neural extension.

15. Related Topics

TF-IDF has near-linear term-frequency weighting; BM25 adds saturation and tunable length normalization. Dense retrieval handles semantic mismatch; hybrid search combines both. Cross-encoder reranking can reorder BM25 candidates. Chunking changes BM25 term statistics and retrieval granularity.

16. Interview Questions

1. **What is BM25?** A lexical ranking function using saturated term frequency, IDF, and length normalization.
2. **What does k_1 control?** How quickly additional term occurrences saturate.
3. **What does b control?** Strength of document-length normalization.
4. **Why IDF?** Rare terms discriminate relevant documents more than common terms.
5. **Why is BM25 fast?** An inverted index visits postings for query terms rather than every document.
6. **BM25 vs TF-IDF?** BM25 explicitly saturates TF and normalizes length with tunable parameters.
7. **BM25 vs dense retrieval?** Exact lexical statistics versus learned semantic similarity.
8. **When does BM25 win?** Codes, proper nouns, numbers, rare terms, and exact phrases.
9. **Does BM25 need training?** No gradient training; parameters/analyzers can be validation-tuned.
10. **What is BM25F?** Field-aware BM25 combining differently weighted title/body/etc. signals.

17. Practice Tasks

- Implement BM25 and compare it with TF-IDF cosine search.
- Sweep k_1 and b on judged queries.
- Build an inverted index rather than scanning all documents.
- Debug analyzer mismatch for hyphenated product codes.
- Combine BM25 ranks with dense ranks using RRF.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Mini Search Engine	Implements postings and BM25	Python standard library	Wikipedia subset	Shows IR fundamentals
Fielded Product Search	Boosts title/brand/model fields	OpenSearch	Amazon products	Production search skill
Lexical RAG Baseline	Measures BM25 before neural additions	Pyserini, FastAPI	BEIR/FiQA	Demonstrates rigorous baselining

19. Quick Revision

- **Key idea:** rare matching terms matter; TF saturates; length is normalized.
- **Main formula:** BM25 term sum with k_1 and b .
- **Metrics:** Recall@k, MRR, nDCG, latency.

- **Common traps:** analyzer mismatch and ignoring exact identifiers.
- **Interview one-liner:** BM25 is the cheap, interpretable lexical baseline that dense RAG systems still need to beat.

20. Final Cheat Sheet

Item	Summary
Input/output	Tokenized query -> ranked documents
Main steps	analyze, postings lookup, TF-IDF-length scoring, top-k
Hyperparameters	k1 , b , analyzer, field weights, top-k
Metrics	Recall@k, MRR, nDCG, query latency
Pros / cons	Fast/exact/interpretable / vocabulary mismatch
Best use cases	Rare terms, identifiers, lexical/hybrid RAG

Dense Retrieval

1. Overview

Dense retrieval uses neural encoders to map queries and passages into a shared low-dimensional vector space and retrieves by vector similarity. Unlike BM25, it can match semantic paraphrases without term overlap. Most dense retrievers are bi-encoders so document vectors can be precomputed and searched with ANN.

2. Intuition

Rather than checking shared words, the model learns that a question and its answer passage should occupy nearby positions—even if they use different vocabulary.

3. Prerequisites

- Transformer encoders, pooling, embeddings, and vector search
- Contrastive learning, negatives, softmax, temperature, backpropagation
- Retrieval metrics and dataset splitting

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Bi-encoder	Independent query/passage encoding	DPR	Scalability
Dual encoder	Separate or partly shared towers	E_q , E_d	Symmetric vs asymmetric
Contrastive training	Positive scores exceed negatives	in-batch softmax	Batch-size effect

Subtopic	Meaning and importance	Example	Interview angle
Hard negatives	Plausible wrong passages	BM25 top hit	Strong signal/false negatives
ANN indexing	Approximate vector top-k	HNSW/IVF	Retrieval recall vs ANN recall
Domain adaptation	Fine-tune on target relevance	legal QA	Embedding drift/versioning

5. Algorithm / Working Process

Training: sample $(q, d+)$ pairs and negatives; encode all items; compute similarity matrix; apply contrastive loss; update encoders; mine harder negatives iteratively. Indexing: encode chunks, normalize as required, store vectors and metadata in ANN. Inference: encode query with the matching model/version, search top-k under filters, return candidates for reranking.

6. Mathematical Foundation

Scores commonly use dot product $s_{ij} = E_q(q_i)^T E_d(d_j)$ or cosine. In-batch negative loss is

$$L = -(1/B) \sum_i \log[\exp(s_{ii}/\tau) / \sum_j \exp(s_{ij}/\tau)].$$

The other batch passages become negatives, giving $B-1$ negatives per query cheaply. A symmetric loss adds passage-to-query cross-entropy. Larger batches provide more negatives but increase compute and false-negative probability.

7. Practical Implementation

```
import torch
import torch.nn.functional as F

# Example normalized embeddings from a query and passage encoder.
query_vectors = F.normalize(torch.randn(4, 384, requires_grad=True), dim=1)
passage_vectors = F.normalize(torch.randn(4, 384, requires_grad=True), dim=1)
temperature = 0.05

logits = query_vectors @ passage_vectors.T / temperature
labels = torch.arange(len(query_vectors)) # diagonal pairs are positives
loss = F.cross_entropy(logits, labels)
loss.backward()
print("contrastive loss:", float(loss.detach()))
```

8. Code Explanation

The matrix contains every query-passage score in the batch. Diagonal entries are labeled positives; off-diagonal passages serve as negatives. Temperature scales score sharpness before cross-entropy. A real model

computes these vectors from tokenized text and optimizes encoder parameters rather than random leaf tensors.

9. Training / Evaluation

Use query-positive pairs with source-disjoint/time-aware splits. Start with random/in-batch negatives, then mine BM25 or previous-model hard negatives and remove likely false negatives. Track validation Recall@k, MRR/nDCG, embedding norms, domain/query slices, and downstream answer quality. Tune encoder, max lengths, prompts, batch size, temperature, learning rate, epochs, hard-negative count, and ANN settings.

10. Complexity and Cost

Training requires two encoder passes and a $B \times B$ score matrix; distributed all-gather can expand negatives. Document encoding is an offline $O(N)$ model cost and vector storage is $O(Nd)$. Query-time cost is one encoder pass plus ANN. GPU helps training/indexing; small models often serve queries on CPU with batching.

11. Common Use Cases

Open-domain QA, semantic enterprise search, multilingual retrieval, code search, recommendations, duplicate detection, and RAG candidate generation.

12. Common Mistakes

- Using random negatives only
- Treating other positives in a batch as negatives
- Mixing model versions between index and query service
- Training on answer text that leaks exact query templates into test
- Using one encoder prefix for both roles when the model expects distinct prompts
- Blaming ANN when encoder Recall@k is already poor under exact search

13. Edge Cases / Limitations

Dense retrieval can miss exact identifiers, numbers, names, and negation. It compresses a passage into one vector, losing fine-grained interactions. Domain and language shift matter. Training labels can contain multiple valid passages, making false negatives common. Reindexing is required after changing the encoder.

14. Variations

- **DPR-style dual encoder:** separate query/context encoders; placement essential.
- **Shared Siamese encoder:** same weights; simple semantic search.
- **Late interaction/ColBERT:** stores token vectors and MaxSim; advanced IR.
- **Multilingual dense retrieval:** shared languages; important for global projects.
- **Generative retrieval:** model generates document IDs; research topic with indexing challenges.

15. Related Topics

Embeddings are the representation, dense retrieval is the supervised IR task and system around them, and vector search executes nearest-neighbor lookup. BM25 offers lexical complement. Fine-tuned retrievers adapt dense geometry to a domain. Cross-encoders trade precomputation for stronger joint interaction.

16. Interview Questions

1. **What is dense retrieval?** Neural bi-encoder retrieval using learned dense query/document vectors.
2. **Why a bi-encoder?** Documents can be encoded offline, enabling scalable ANN.
3. **What is in-batch negative training?** Other batch passages serve as negatives for each query.
4. **Why hard negatives?** They teach distinctions the model currently confuses.
5. **Risk of hard negatives?** Some may actually be relevant, producing harmful false-negative gradients.
6. **Why reindex after fine-tuning?** Document vectors must share the updated geometry with query vectors.
7. **Dense vs cross-encoder?** Dense is scalable independent encoding; cross-encoder is slower joint scoring and usually more accurate.
8. **How isolate ANN quality?** Compare ANN results with exact search over the same embeddings.
9. **What does temperature do?** Controls sharpness of contrastive similarity logits.
10. **How combine with BM25?** Fuse candidate ranks/scores, commonly with RRF, then rerank.

17. Practice Tasks

- Implement in-batch contrastive loss.
- Fine-tune a SentenceTransformer on query-passage pairs.
- Compare random, BM25, and mined hard negatives.
- Separate encoder recall from ANN recall.
- Test dense vs BM25 on paraphrase and identifier slices.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Domain Retriever	Fine-tunes and indexes domain passages	PyTorch, SentenceTransformers, FAISS	FiQA or SciFact	End-to-end neural IR
Multilingual QA Search	Cross-language passage retrieval	multilingual-e5, Qdrant	MIRACL	Strong research/project value
Hard-Negative Study	Compares mining curricula	HF, PyTorch, BEIR	MS MARCO/BEIR	Demonstrates training insight

19. Quick Revision

- **Key idea:** learn query-passage geometry, then search vectors.
- **Main formula:** contrastive softmax over positives and negatives.
- **Metrics:** Recall@k, MRR/nDCG, ANN recall, latency.
- **Common traps:** weak/false negatives and stale index vectors.
- **Interview one-liner:** dense retrieval gains semantic recall by learning a bi-encoder space whose documents remain precomputable.

20. Final Cheat Sheet

Item	Summary
Input/output	Query -> semantically ranked passage IDs
Main steps	train encoders, embed corpus, ANN query, return candidates
Hyperparameters	encoder, dimension, temperature, negatives, max lengths, k
Metrics	Recall@k, MRR, nDCG, ANN recall, p95 latency
Pros / cons	Paraphrase matching / exact-term weakness and reindexing
Best use cases	Semantic RAG and natural-language search

Cross-Encoder Reranker

1. Overview

A cross-encoder reranker jointly encodes a query and one candidate passage, then predicts a relevance score. Full token-to-token attention captures exact alignment, negation, and fine-grained evidence better than independent embeddings. Because it must run once per candidate, it is used after high-recall retrieval rather than over the full corpus.

2. Intuition

A bi-encoder compares two prewritten summaries of a query and passage. A cross-encoder lays both texts side by side and reads them together before judging relevance.

3. Prerequisites

- Transformer self-attention and special-pair tokenization
- Binary classification, logits, cross-entropy, pairwise ranking losses
- Candidate retrieval, nDCG/MRR, batching, and truncation

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Joint input	[CLS] query [SEP] passage [SEP]	BERT pair	Why no precomputed doc score
Interaction	Every query token attends to passage tokens	negation matching	Accuracy advantage
Relevance head	CLS state -> scalar/logits	relevant score	Logit vs probability
Hard negatives	Retrieved but wrong passages	same topic, wrong rule	Training distribution

Subtopic	Meaning and importance	Example	Interview angle
Truncation	Pair must fit max length	512 tokens	Preserve answer-bearing span
Distillation	Teacher cross-encoder trains smaller models	teacher logits	Deployment optimization

5. Algorithm / Working Process

Training: create query, passage, relevance tuples or preferences; jointly tokenize; run Transformer; map pooled state to a score; optimize pointwise/pairwise/listwise loss. Inference: retrieve K candidates; batch K query-passage pairs; predict scores; sort; select final k. Monitor input truncation and dynamic batch sizes.

6. Mathematical Foundation

Let $h_{CLS} = \text{Transformer}([q;d])$ and $s = w^T h_{CLS} + b$. Pointwise binary loss is BCE on $\text{sigma}(s)$. Pairwise RankNet loss is $-\log \text{sigma}(s_+ - s_-)$. For listwise softmax with one positive among candidates, $L = -\log \frac{\exp(s_+)}{\sum_j \exp(s_j)}$. nDCG aligns evaluation with graded top-order quality better than accuracy on independently sampled pairs.

7. Practical Implementation

```
from sentence_transformers import CrossEncoder

model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2", max_length=256)
query = "Can unused leave move to next year?"
passages = [
    "Up to five annual-leave days can be carried into the next year.",
    "Employees should submit leave requests two weeks in advance.",
]

scores = model.predict([(query, p) for p in passages], batch_size=16)
best = max(range(len(passages)), key=lambda i: scores[i])
print(passages[best], float(scores[best]))
```

8. Code Explanation

The library builds paired Transformer inputs and returns one score per pair. `max_length` limits total query-plus-passage tokens. `batch_size` controls throughput/memory. Raw scores rank candidates; probabilities require task-specific calibration and are unnecessary for simple sorting.

9. Training / Evaluation

Use positives and hard negatives drawn from the deployed retriever. Separate sources and near-duplicate queries across splits. Evaluate nDCG@10, MRR, Precision@k, pairwise accuracy, downstream answer quality, p95 latency, throughput, and truncation rate. Tune model size, max length, candidate depth, batch size, loss, learning rate, and negative ratios.

10. Complexity and Cost

Each of K pairs performs Transformer attention over combined length L : roughly $O(KL^2)$. Passage representations cannot be reused across different queries. GPU batching greatly improves throughput, but tail latency and memory limit batch size. Smaller distilled MiniLM models are common; large LLM rerankers cost more but may improve complex judgments.

11. Common Use Cases

RAG evidence ranking, web/product search, legal passage ranking, support knowledge selection, scientific retrieval, and training/distilling dense retrievers.

12. Common Mistakes

- Using a cross-encoder as a full-corpus retriever
- Training with random negatives unlike inference candidates
- Truncating from the wrong side and dropping evidence
- Applying sigmoid and assuming calibrated confidence
- Reporting classification accuracy rather than ranking metrics
- Changing the retriever without refreshing reranker evaluation

13. Edge Cases / Limitations

The model cannot score what first-stage retrieval missed. Long tables or passages exceed sequence length. Input order, domain language, and multilinguality can shift scores. Independent scoring does not enforce diversity across selected candidates. Cross-encoder output can still prefer topical but factually wrong or stale text.

14. Variations

- **Pointwise cross-encoder:** score each pair; standard placement implementation.
- **Pairwise reranker:** compare two candidates; stronger relative reasoning, more calls.
- **Listwise LLM reranker:** order a whole list; advanced and expensive.
- **Distilled reranker:** smaller student trained from larger teacher; deployment-friendly.
- **MonoT5/sequence-to-sequence:** generates relevance tokens; important in neural IR literature.

15. Related Topics

Dense retrievers use independent embeddings for candidate speed; cross-encoders add joint interaction for accuracy. Reranking is the pipeline stage; cross-encoder is one implementation. Hard negatives connect both: a cross-encoder can label or distill data for fine-tuned retrievers.

16. Interview Questions

1. **What is a cross-encoder?** A model jointly encoding query and document to predict relevance.
2. **Why more accurate than bi-encoder?** It models all token-level cross-interactions before scoring.
3. **Why not index cross-encoder outputs?** The passage representation/score depends on the current query.
4. **Where is it used?** After a fast retriever on tens or hundreds of candidates.

5. **What loss is common?** Pointwise BCE, pairwise RankNet, or listwise softmax.
6. **Why hard negatives?** They match the difficult candidate distribution seen at inference.
7. **What metric matters?** nDCG/MRR/Precision at final ranks, plus latency.
8. **How reduce latency?** Smaller/distilled model, batching, lower depth/length, conditional routing.
9. **How handle long passages?** Better chunks, sliding windows with max score, or targeted span extraction.
10. **Are scores probabilities?** Not without calibration; raw logits are suitable for ranking.

17. Practice Tasks

- Rerank dense top-100 on a BEIR dataset.
- Fine-tune pointwise and pairwise variants.
- Plot quality/latency by model size and depth.
- Analyze queries hurt by truncation.
- Distill cross-encoder scores into a bi-encoder.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Neural Reranking Service	Batched low-latency scoring API	PyTorch, HF, FastAPI	MS MARCO	Production DL serving
Reranker Distillation	Compresses a large relevance teacher	SentenceTransformers	BEIR	Research and optimization
Legal Passage Reranker	Learns domain relevance	LegalBERT, FAISS	COLIEE	Domain adaptation signal

19. Quick Revision

- **Key idea:** joint token interaction for precise relevance.
- **Main formula:** scalar CLS score with pairwise/listwise loss.
- **Metrics:** nDCG, MRR, Precision@k, p95 latency.
- **Common traps:** full-corpus use, truncation, random negatives.
- **Interview one-liner:** a cross-encoder spends per-query computation on a shortlist to recover interactions a precomputed embedding must compress away.

20. Final Cheat Sheet

Item	Summary
Input/output	Query-passage pair -> relevance score
Main steps	pair-tokenize, joint encode, score, sort
Hyperparameters	model, max length, depth K, batch, loss
Metrics	nDCG/MRR, final Precision@k, latency
Pros / cons	High accuracy / K model passes, no precompute

Item	Summary
Best use cases	Second-stage ranking for valuable top results

Query Rewriting

1. Overview

Query rewriting transforms a user's raw request into one or more retrieval-friendly queries. It resolves conversational references, fixes vocabulary mismatch, expands acronyms, decomposes compound questions, or generates a hypothetical answer. Good rewriting improves recall; uncontrolled rewriting can change intent and retrieve confidently wrong evidence.

2. Intuition

"What about its battery?" is useless without conversation history. A rewriter turns it into "What is the battery life of the ThinkPad X1 Carbon?" before retrieval.

3. Prerequisites

- Search queries, conversational state, entities, and intent classification
- Sparse/dense retrieval and query expansion
- Sequence-to-sequence/LLM prompting and evaluation

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Standalone rewrite	Resolve coreference/history	"its" -> product name	Preserve intent
Expansion	Add synonyms/entities	SSO -> single sign-on	Helps lexical recall
Multi-query	Generate diverse formulations	3 paraphrases	Recall vs cost/noise
Decomposition	Split multiple information needs	eligibility + deadline	Multi-hop/compound queries
HyDE	Embed hypothetical answer text	imagined relevant passage	Why it can help dense search
Guardrails	Preserve entities/constraints	dates, negation, tenant	Prevent query drift

5. Algorithm / Working Process

1. Classify whether the raw query is already retrieval-ready.
2. Gather only necessary trusted conversation state and entity metadata.
3. Rewrite to a standalone query while preserving names, numbers, negation, and constraints.
4. Optionally create expansions, subqueries, or a hypothetical document.

5. Retrieve per query and fuse/deduplicate results.
6. Validate that final evidence answers the original—not merely rewritten—query.
7. Log raw and rewritten forms for evaluation and debugging.

6. Mathematical Foundation

A seq2seq rewriter models $P(q' | q, h) = \text{product}_t P(q'_t | q, h, q'_{<t})$. Multi-query retrieval returns the union or fused ranking $A = \text{Fuse}(R(q'_1), \dots, R(q'_m))$. The useful objective is downstream retrieval gain, e.g. $\Delta \text{Recall}@k$, penalized by cost and intent drift—not text similarity between original and rewritten query.

7. Practical Implementation

```
def make_standalone(query: str, conversation_entities: dict[str, str]) -> str:
    """Deterministic rewrite for known references; LLM fallback can handle harder
    cases."""
    rewritten = query
    for pronoun, entity in conversation_entities.items():
        rewritten = rewritten.replace(pronoun, entity)
    return rewritten

raw = "What is its battery life?"
standalone = make_standalone(raw, {"its": "the ThinkPad X1 Carbon's"})
assert standalone == "What is the ThinkPad X1 Carbon's battery life?"
print(standalone)
```

8. Code Explanation

The example demonstrates the safest case: conversation state has already resolved a reference to a known entity, and replacement preserves the question. Real coreference is more complex; an LLM rewrite should receive explicit preservation rules and return structured output. Always retain the raw query for final answer validation.

9. Training / Evaluation

Data can include conversational query/standalone-query pairs, search reformulations, and query-relevant document labels. Split by conversation/entity/topic. Measure downstream Recall@k/nDCG, intent-preservation error, entity/constraint retention, latency, and token cost. Compare no rewrite, deterministic rewrite, and LLM rewrite. Human review is crucial for silent drift.

10. Complexity and Cost

Rule-based normalization is negligible. An LLM rewrite adds one generation call before retrieval; multi-query multiplies search and reranking candidates. Run multiple searches concurrently and cap query count. Cache only context-independent rewrites; conversation-dependent or permission-dependent inputs need careful keys.

11. Common Use Cases

Conversational RAG, acronym-heavy enterprise search, multilingual search, vague user queries, complex question decomposition, and improving retrieval over short or underspecified queries.

12. Common Mistakes

- Rewriting every query even when exact terms are already ideal
- Dropping negation, numbers, dates, or quoted identifiers
- Answering the rewritten query rather than the original intent
- Generating many near-identical queries and multiplying cost
- Letting a rewrite invent entities or assumptions
- Evaluating fluency rather than retrieval gains and drift

13. Edge Cases / Limitations

Ambiguous references may need user clarification rather than guessing. Query expansion can broaden to irrelevant meanings. HyDE can inject false specifics. Conversation history may contain prompt injection or stale entity state. Exact keyword queries can be harmed by paraphrasing.

14. Variations

- **Normalization/spell correction:** low-risk lexical cleanup; placement basic.
- **Conversational standalone rewrite:** resolves references; common project requirement.
- **Multi-query expansion:** improves recall by diverse formulations; advanced practical pattern.
- **HyDE:** embeds a hypothetical answer to search passages; research/interview-worthy.
- **Decomposition:** emits subquestions for compound/multi-hop tasks; required for multi-hop/agentic RAG.

15. Related Topics

Hybrid search can route expansions to lexical and dense indexes. Multi-hop RAG uses decomposition and iterative rewrites. Agentic RAG lets a model decide when/how to reformulate. Metadata filtering preserves constraints that natural-language rewriting alone should not encode insecurely.

16. Interview Questions

1. **Why rewrite queries?** To resolve context and better match how relevant information is expressed/indexed.
2. **Main risk?** Semantic drift that changes the user's actual information need.
3. **What is a standalone query?** A rewrite understandable without prior conversation.
4. **What is multi-query retrieval?** Search several reformulations, then fuse their candidates.
5. **What is HyDE?** Generate a hypothetical relevant passage and embed it as the search query.
6. **Why can HyDE work?** Answer-like text may align better with document embedding distribution than a short question.
7. **When not rewrite?** Exact IDs, quoted phrases, or already clear queries where transformation adds risk/cost.
8. **How evaluate?** Downstream recall/ranking plus entity/constraint preservation and drift rate.
9. **How handle ambiguity?** Use verified context or ask the user rather than inventing a referent.
10. **How control cost?** Route only difficult queries, limit variants, parallelize retrieval, and deduplicate early.

17. Practice Tasks

- Build a standalone-query dataset from multi-turn conversations.
- Compare raw, rewritten, and multi-query Recall@10.
- Create tests ensuring names, numbers, and negation survive.
- Diagnose a HyDE query that invents the wrong product.
- Implement query routing that skips rewrite for exact identifiers.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Conversational Search Rewriter	Resolves follow-up references	T5/HF, FAISS	QReCC	Strong conversational IR project
Multi-query RAG	Generates/fuses diverse searches	LLM, BM25, vector DB	HotpotQA	Advanced retrieval pipeline
Query Drift Auditor	Detects lost entities/constraints	Python, NER, eval UI	Custom conversations	Reliability-focused engineering

19. Quick Revision

- **Key idea:** transform user language into retrieval language without changing intent.
- **Main formula:** conditional rewrite likelihood; evaluate downstream **Delta Recall@k**.
- **Metrics:** Recall/nDCG gain, drift, constraint retention, latency.
- **Common traps:** inventing intent and rewriting exact queries.
- **Interview one-liner:** query rewriting improves the retriever's input, but the original query remains the contract the answer must satisfy.

20. Final Cheat Sheet

Item	Summary
Input/output	Raw query + trusted context -> standalone/expanded queries
Main steps	classify, resolve, preserve, rewrite, retrieve, fuse, validate
Hyperparameters	variant count, prompt/model, history window, routing threshold
Metrics	Recall@k gain, nDCG, drift/retention, cost
Pros / cons	Better recall/context resolution / drift and added latency
Best use cases	Conversational, vague, compound, vocabulary-mismatched queries

Metadata Filtering

1. Overview

Metadata filtering restricts retrieval using structured attributes such as tenant, permissions, document type, language, region, product, date, or version. It improves relevance and is a security boundary when metadata represents authorization. Filters should be enforced by the search backend before or during candidate generation, not merely removed after content has already been exposed.

2. Intuition

A library search for "leave policy" should first enter the correct company's authorized room, then search its shelves. Searching every room and hiding unauthorized books afterward is both inefficient and unsafe.

3. Prerequisites

- Structured data types, Boolean predicates, database indexes, and schemas
- Vector/sparse search, ANN candidate selection, and access control
- Query parsing, entity extraction, dates, and evaluation slices

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Hard filter	Mandatory predicate	<code>tenant_id=42</code>	Security cannot be soft boost
Soft boost	Preference, not exclusion	newer documents score higher	Filter vs ranking feature
Pre-filter	Restrict search universe first	ACL bitmap before ANN	Correctness/security
Post-filter	Filter returned candidates	discard after top-100	Can starve results
Schema/cardinality	Metadata type and value distribution	date vs unique user ID	Index performance
Filter extraction	Turn natural language into structured predicate	"after 2025"	Validation and ambiguity

5. Algorithm / Working Process

1. During ingestion, validate metadata types and attach stable tenant, ACL, source, time, version, and content attributes to every chunk.
2. Derive trusted mandatory filters from authenticated identity and server-side policy.
3. Parse optional user constraints into a validated structured representation.
4. Execute filters inside sparse/vector candidate generation where supported.
5. Adapt ANN effort if the eligible subset is small.
6. Recheck authorization before context construction and generation.
7. Log filter predicates and counts without leaking sensitive values.

6. Mathematical Foundation

For predicate $F(d) \in \{0,1\}$, filtered retrieval is

$\arg \text{top-k}_{\{d \in D, F(d)=1\}} s(q,d).$

Post-filtering a global top-K approximates this as $\{d \in \text{topK_global}: F(d)=1\}$ and is not equivalent: eligible high-scoring items outside global K are missed. Filter selectivity is $\sigma = |\{d: F(d)=1\}| / |D|$; low selectivity can require specialized filtered indexes or higher ANN exploration.

7. Practical Implementation

```
from datetime import date

records = [
    {"id": "a", "tenant": 7, "year": 2026, "score": 0.81},
    {"id": "b", "tenant": 8, "year": 2026, "score": 0.99},
    {"id": "c", "tenant": 7, "year": 2024, "score": 0.95},
]

def filtered_top_k(records, *, tenant: int, min_year: int, k: int):
    eligible = (r for r in records
                if r["tenant"] == tenant and r["year"] >= min_year)
    return sorted(eligible, key=lambda r: r["score"], reverse=True)[:k]

result = filtered_top_k(records, tenant=7, min_year=2025, k=2)
assert [r["id"] for r in result] == ["a"]
print(result)
```

8. Code Explanation

Filtering occurs before top-k sorting, so unauthorized or out-of-date records never compete. Keyword-only arguments reduce accidental parameter swaps. In production, the database/search engine executes an equivalent typed predicate and an authenticated service—not the LLM—supplies the tenant/ACL constraint.

9. Training / Evaluation

Filtering itself is rule-driven. If an LLM extracts optional filters, build query-to-filter examples and evaluate exact predicate accuracy by field/operator/value. Test authorization with allow/deny matrices and adversarial attempts. Report filtered Recall@k, empty-result rate, p95 latency by selectivity, filter extraction precision/recall, and zero unauthorized retrieval/context events.

10. Complexity and Cost

Indexed scalar filters can use bitmaps/B-trees efficiently. Vector pre-filter performance varies: a very small allowed subset may fragment HNSW traversal or require brute force over eligible IDs. High-cardinality ACL lists add storage and update cost. Separate indexes per tenant simplify isolation but multiply operational overhead for many small tenants.

11. Common Use Cases

Multi-tenant enterprise RAG, role-based knowledge access, date/version-scoped policy search, product/category search, geographic search, language routing, and regulatory data residency.

12. Common Mistakes

- Letting the LLM decide mandatory authorization filters
- Post-filtering a small global top-k
- Treating a hard access constraint as a score penalty
- Storing inconsistent date/string types
- Failing to propagate metadata to every derived chunk
- Omitting filter/selectivity cases from retrieval benchmarks

13. Edge Cases / Limitations

Permissions can change faster than indexes refresh. Missing metadata needs explicit deny/default behavior. Nested groups and document-level exceptions make ACLs complex. Time zones and effective-date intervals cause boundary bugs. Extremely selective filters can hurt ANN recall or latency.

14. Variations

- **Scalar pre-filter:** equality/range before search; placement essential.
- **Bitmap ACL filter:** efficient set intersection; strong enterprise pattern.
- **Partitioned indexes:** tenant/category-specific index; useful for large isolated groups.
- **Attribute boosting:** soft relevance feature rather than exclusion.
- **Self-query retrieval:** LLM produces query plus filter AST; useful for natural constraints but requires validation.

15. Related Topics

Metadata originates during ingestion/chunking. Hybrid search must apply the same predicates to both branches. Vector search engines differ in filtered ANN behavior. Context construction rechecks authorization. Query rewriting may identify optional constraints but cannot override trusted identity policy.

16. Interview Questions

1. **What is metadata filtering?** Restricting candidate documents by structured predicates alongside relevance search.
2. **Pre- vs post-filter?** Pre-filter searches only eligible documents; post-filter discards from a global shortlist and may miss valid results.
3. **Why is ACL a hard filter?** Unauthorized content must never be exposed regardless of relevance score.
4. **Who supplies tenant identity?** Trusted authenticated server context, not user text or LLM output.
5. **What is selectivity?** Fraction of corpus satisfying a predicate.
6. **Why can selective filters hurt ANN?** Graph/partition traversal may have few eligible neighbors and need more exploration.
7. **How handle missing ACL metadata?** Fail closed: treat content as unauthorized until explicitly classified.
8. **What is self-query retrieval?** A model converts natural language into text query plus structured filter expression.
9. **How test security?** Explicit allow/deny matrices, adversarial tenant queries, and checks at retrieval/context boundaries.
10. **Filter vs boost?** A filter excludes; a boost changes ordering but keeps candidates eligible.

17. Practice Tasks

- Implement equality, range, and list membership filters.
- Demonstrate why post-filtered top-k differs from true filtered top-k.
- Create tenant authorization regression tests.
- Benchmark filtered ANN across selectivity buckets.
- Parse natural-language dates into a validated filter AST.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Secure Multi-tenant RAG	Enforces server-derived ACLs	FastAPI, PostgreSQL/pgvector	Synthetic enterprise docs	Strong security architecture
Temporal Policy Search	Filters by effective intervals/version	OpenSearch, Python	Government policy archive	Shows temporal data handling
Self-query Catalog	Extracts validated price/category filters	Pydantic, LLM, vector DB	Amazon products	Structured LLM integration

19. Quick Revision

- **Key idea:** define the eligible corpus before ranking.
- **Main formula:** top-k under predicate $F(d)=1$.
- **Metrics:** filtered Recall@k, unauthorized hits, latency by selectivity.
- **Common traps:** post-filter starvation and LLM-controlled ACLs.
- **Interview one-liner:** relevance decides what is useful; trusted metadata filters decide what is eligible and safe to retrieve.

20. Final Cheat Sheet

Item	Summary
Input/output	Query + trusted predicates -> eligible ranked records
Main steps	validate schema, derive ACL, pre-filter, search, recheck
Hyperparameters	indexed fields, filter mode, ANN effort, partitions
Metrics	filtered recall, empty rate, security violations, latency
Pros / cons	Relevance and isolation / schema and ANN complexity
Best use cases	Multi-tenant, time/version/category-scoped RAG

Multi-hop RAG

1. Overview

Multi-hop RAG answers questions requiring evidence from multiple facts or documents where later retrieval depends on earlier findings. A single query often cannot retrieve all evidence because an intermediate entity or relation is missing. The system decomposes the task, retrieves iteratively, and combines a traceable evidence chain.

2. Intuition

To answer "Which country hosts the company that created product X?" first find product X's creator, then find that company's country. The second search depends on the first result.

3. Prerequisites

- Single-hop RAG, query rewriting/decomposition, and entity linking
- Graph traversal intuition, state management, and stopping conditions
- Evidence aggregation, citations, and multi-step evaluation

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Decomposition	Split question into dependent subquestions	product -> company -> country	Sequential vs parallel
Bridge entity	Intermediate answer enabling next hop	company name	Entity disambiguation
Iterative retrieval	Search conditioned on accumulated state	hop-2 query	Error propagation
Evidence chain	Ordered sources supporting reasoning	S1 then S2	Chain completeness
Stopping	Decide enough evidence or budget exhausted	answerable after 2 hops	Loop prevention
State	Store facts, provenance, unresolved needs	structured scratchpad	Avoid unsupported memory

5. Algorithm / Working Process

1. Classify whether the question needs multiple hops.
2. Decompose it into the first answerable subquestion and remaining dependency.
3. Retrieve/rerank evidence and extract a bridge entity with provenance.
4. Validate/disambiguate the entity.
5. Form the next subquery from the original goal plus verified state.
6. Repeat until all required facts are supported or hop/cost limits are reached.
7. Construct a context preserving the chain and generate an answer citing every hop.
8. Abstain if a required link is missing or contradictory.

6. Mathematical Foundation

Let latent evidence chain be $Z=(z_1, \dots, z_H)$. A conceptual factorization is

$$P(y, Z|q) = P(z_1|q) \prod_{h=2}^H P(z_h|q, z_{<h}) * P(y|q, Z).$$

If hop success probabilities are approximately p_h , complete-chain success is roughly $\prod_h p_h$, showing error compounding. Path Recall measures whether all gold supporting documents are found; supporting-fact F1 and joint answer/evidence metrics prevent credit for lucky unsupported answers.

7. Practical Implementation

```
knowledge = {
    "Who created Product X?": "Acme Labs",
    "Where is Acme Labs headquartered?": "India",
}

def two_hop_answer(product: str):
    creator_q = f"Who created {product}?"
    creator = knowledge.get(creator_q)
    if not creator:
        return "Insufficient evidence", []
    country_q = f"Where is {creator} headquartered?"
    country = knowledge.get(country_q)
    if not country:
        return "Insufficient evidence", [creator_q]
    return country, [creator_q, country_q]

answer, trace = two_hop_answer("Product X")
assert answer == "India" and len(trace) == 2
print(answer, trace)
```

8. Code Explanation

The first lookup returns a bridge entity, which is inserted into the second query. The function refuses to guess if either hop fails and returns a trace of evidence requests. Real systems replace the dictionary with retriever/reranker/extractor calls and store source IDs plus confidence for each fact.

9. Training / Evaluation

Datasets need questions, final answers, supporting passages/facts, and ideally decomposition traces. Split by bridge entities/documents to test generalization. Evaluate per-hop Recall@k, complete path recall, supporting-fact F1, answer EM/F1, joint answer-plus-evidence, hops, latency, and cost. Tune max hops, per-hop top-k, beam/path width, entity confidence, and stop thresholds.

10. Complexity and Cost

Sequential hops add latency approximately as a sum of retrieval/generation steps. Branching b possible entities over depth H can grow $O(b^H)$ without pruning. Cache reusable subqueries, parallelize independent branches, cap hops/paths, and use cheap extraction where possible. Longer evidence chains increase context and verification cost.

11. Common Use Cases

Cross-document QA, company/person relationship research, biomedical literature synthesis, legal precedent chains, troubleshooting dependency graphs, and questions involving comparison or temporal sequences.

12. Common Mistakes

- Treating every complex question as one broad vector query
- Generating later subqueries from unverified model guesses
- Losing source provenance for bridge facts
- Allowing unbounded loops or branching
- Evaluating only final answer and rewarding lucky guesses
- Decomposing independent subquestions sequentially instead of in parallel

13. Edge Cases / Limitations

Ambiguous bridge entities can send retrieval down the wrong path. One missing link breaks the full chain. Conflicting facts create multiple plausible paths. Some questions require aggregation over many items rather than a short chain. Sequential latency is high and generated reasoning traces are not guaranteed faithful.

14. Variations

- **Fixed decomposition:** known sequence of subquestions; best simple project version.
- **Iterative retrieve-read-retrieve:** next query derived from evidence; core advanced pattern.
- **Beam search over evidence paths:** retains alternatives; research-relevant.
- **Graph-guided multi-hop:** traverses explicit relations; useful when entities are reliable.
- **Parallel decomposition:** retrieves independent facets concurrently; useful for comparisons.

15. Related Topics

Query rewriting creates subqueries; Graph RAG supplies explicit paths and neighborhoods; Agentic RAG plans tools/hops dynamically. Context construction must preserve chain provenance. Cross-encoder reranking and entity linking reduce hop errors.

16. Interview Questions

1. **What makes a question multi-hop?** Its answer requires combining dependent facts from multiple evidence items.
2. **What is a bridge entity?** An intermediate entity found in one hop and used to retrieve the next.
3. **Why not retrieve once?** Later evidence may not share terms with the original query and requires discovered context.
4. **Main failure mode?** Early-hop error propagation through subsequent queries.
5. **How stop loops?** Max hops/cost, repeated-query/state detection, and an evidence-sufficiency decision.
6. **How evaluate?** Per-hop recall, full-path recall, supporting-fact F1, and joint answer-evidence correctness.
7. **Sequential vs parallel decomposition?** Sequential when dependencies exist; parallel for independent facets.
8. **How handle ambiguous bridge entities?** Link/disambiguate using context or keep a bounded beam of alternatives.

9. **Why preserve provenance per hop?** The final answer must prove every link, not cite unrelated final text.
10. **Multi-hop vs agentic RAG?** Multi-hop describes dependent retrieval; agentic RAG adds dynamic planning/tool control and may perform multi-hop retrieval.

17. Practice Tasks

- Build a deterministic two-hop retriever over linked Wikipedia passages.
- Measure product-of-hop error as depth increases.
- Add entity disambiguation and alternative path beams.
- Debug a correct final answer supported by an incomplete chain.
- Parallelize comparison questions with two independent branches.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Evidence-Chain QA	Retrieves and cites two-hop paths	HF, FAISS, FastAPI	HotpotQA	Canonical advanced RAG project
Company Researcher	Links products, companies, and countries	spaCy, vector DB, APIs	Wikidata snapshots	Entity/retrieval integration
Multi-hop Failure Lab	Visualizes per-hop errors and beams	Streamlit, Python	2WikiMultiHopQA	Research-quality analysis

19. Quick Revision

- **Key idea:** retrieve, verify a bridge fact, then retrieve again.
- **Main formula:** chain factorization; success compounds across hops.
- **Metrics:** hop/path recall, supporting-fact F1, joint answer-evidence.
- **Common traps:** unverified bridge entities and unbounded loops.
- **Interview one-liner:** multi-hop RAG turns retrieval into a dependency-aware evidence chain rather than one top-k lookup.

20. Final Cheat Sheet

Item	Summary
Input/output	Complex query -> answer plus ordered evidence chain
Main steps	classify, decompose, retrieve, extract, verify, repeat, synthesize
Hyperparameters	max hops, per-hop k, beam width, confidence/stop thresholds
Metrics	per-hop recall, path recall, supporting F1, joint accuracy
Pros / cons	Solves compositional questions / error and latency compound
Best use cases	Dependent cross-document reasoning

Graph RAG

1. Overview

Graph RAG augments retrieval with an explicit graph of entities, concepts, documents, or claims connected by relations. It can traverse multi-hop relationships, retrieve structured neighborhoods, and use community summaries for global corpus questions. Graph RAG is valuable when relationships—not isolated passages—carry the answer, but graph extraction and maintenance add substantial complexity.

2. Intuition

Vector RAG finds paragraphs that look similar to a question. Graph RAG also follows arrows: person -> works_at -> company -> located_in -> country, retaining why the items are connected.

3. Prerequisites

- Graphs, nodes, edges, paths, neighborhoods, centrality, and communities
- Entity/relation extraction and entity resolution
- RAG, multi-hop retrieval, databases, provenance, and graph queries

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Entity node	Canonical real-world object	Acme_Labs	Entity resolution
Relation edge	Typed connection with provenance	created(ProductX)	Extraction confidence
Claim/document node	Preserves source evidence	sentence/page ID	Avoid graph as unsupported truth
Neighborhood/path	Related subgraph for a query	2-hop ego graph	Traversal explosion
Community summary	Summary of densely connected region	product ecosystem	Global questions
Graph + vector index	Semantic entry point plus traversal	entity embedding -> neighbors	Complementary retrieval

5. Algorithm / Working Process

1. Parse documents and extract entity mentions, relations, claims, and source locations.
2. Resolve mentions to canonical entities; retain aliases and confidence.
3. Store typed edges with timestamps and provenance.
4. Optionally detect communities and create source-grounded summaries.
5. At query time, identify/link seed entities or retrieve seed nodes semantically.

6. Traverse bounded relation types/depths or execute a graph query.
7. Rank paths/subgraphs by relevance, confidence, authority, and recency.
8. Fetch original passages for selected edges and construct cited context.
9. Generate an answer that distinguishes sourced facts from inferred connections.

6. Mathematical Foundation

A graph is $G=(V,E)$, possibly with typed edges (u,r,v) . A length- H path is $(v_0, r_1, v_1, \dots, r_H, v_H)$. Personalized PageRank can score nodes:

$$p_{t+1} = (1-\alpha)s + \alpha P^T p_t,$$

where s concentrates probability on query seeds. Path scoring may combine $\text{sum edge_confidence}$, semantic relevance, and length penalties. Graph embeddings often train with a margin or logistic loss so true triples score above corrupted triples.

7. Practical Implementation

```
from collections import defaultdict, deque

graph = defaultdict(list)
graph["Product X"].append(("created_by", "Acme Labs", "S1"))
graph["Acme Labs"].append(("headquartered_in", "India", "S2"))

def paths(start, max_hops=2):
    queue = deque([(start, [])])
    while queue:
        node, path = queue.popleft()
        if path:
            yield path
        if len(path) < max_hops:
            for relation, neighbor, source in graph[node]:
                queue.append((neighbor, path + [(node, relation, neighbor,
source)]))

found = list(paths("Product X", max_hops=2))
assert found[-1][-1][2] == "India"
print(found[-1])
```

8. Code Explanation

The adjacency list stores typed edges and a source ID, making every relationship traceable. Breadth-first traversal enumerates bounded paths and avoids unbounded depth. A production graph needs cycle checks, entity resolution, relation constraints, confidence, temporal validity, authorization, and ranking before source passages reach the LLM.

9. Training / Evaluation

Evaluate entity precision/recall, entity-linking accuracy, relation/edge precision/recall, path recall, source provenance correctness, answer accuracy/faithfulness, and latency. Split by entities/documents/time to avoid memorizing graph facts. For community summaries, verify claim coverage and citation support. Tune extraction thresholds, traversal depth, relation allowlists, seed count, path width, and summary granularity.

10. Complexity and Cost

Extraction can require model calls over every chunk and entity resolution across the corpus. A naive traversal has $O(b^H)$ nodes for branching factor b and depth H ; bounded relation-aware search is essential. Storage includes graph, aliases, provenance, original chunks, and often a vector index. Incremental updates and deletion propagation are harder than ordinary chunk indexes.

11. Common Use Cases

Organizational knowledge, supply-chain analysis, biomedical entity relations, fraud networks, legal citations, research landscape summaries, dependency analysis, and corpus-wide thematic questions.

12. Common Mistakes

- Building a graph when independent passage retrieval already solves the task
- Treating extracted edges as truth without source provenance/confidence
- Failing entity resolution and creating duplicate nodes
- Traversing all relation types/depths indiscriminately
- Generating summaries that cannot cite original evidence
- Ignoring temporal validity, access control, and delete propagation

13. Edge Cases / Limitations

Entity extraction struggles with implicit relations, tables, aliases, and domain terminology. Incorrect edges create convincing false paths. Dense, highly connected hubs cause traversal explosion. Global community summaries may become stale or lossy. Some corpora have little meaningful graph structure and do not justify the overhead.

14. Variations

- **Knowledge-graph RAG:** query curated entities/relations; important for structured domains.
- **Document graph RAG:** nodes are chunks/docs linked by citations or similarity; practical lighter variant.
- **Community-summary Graph RAG:** hierarchical summaries for global questions; advanced/research relevant.
- **Graph neural retrieval:** learns node/path relevance; research-focused.
- **Graph + vector hybrid:** semantic seeds followed by structured traversal; common pragmatic design.

15. Related Topics

Multi-hop RAG builds evidence chains dynamically; Graph RAG makes relationships explicit and traversable. Knowledge graphs require entity resolution and schema design. Dense retrieval provides semantic entry points. Agentic RAG may choose graph queries as one of several tools.

16. Interview Questions

1. **What is Graph RAG?** RAG that retrieves graph entities, relations, paths, or community summaries plus source evidence.
2. **When is it better than vector RAG?** Relationship-heavy, multi-hop, or global corpus questions.
3. **What is entity resolution?** Mapping different mentions/aliases to one canonical node.
4. **Why store provenance on edges?** Extracted relations need auditable supporting text.
5. **How retrieve from a graph?** Link seed entities, traverse constrained neighborhoods/paths, rank, then fetch supporting passages.
6. **What causes traversal explosion?** Branching factor grows candidate nodes exponentially with depth.
7. **How control it?** Limit depth, relation types, seed/path width, confidence, and visited nodes.
8. **What are community summaries?** Grounded summaries of graph clusters used for corpus-wide questions.
9. **Main operational challenge?** Accurate extraction/entity resolution and consistent updates/deletes across graph and source indexes.
10. **Must an LLM answer from graph triples alone?** Prefer original supporting passages for nuance, verification, and citations.

17. Practice Tasks

- Extract entities/relations from 100 articles and inspect errors.
- Implement bounded BFS with cycle detection and relation filters.
- Compare vector-only and graph-assisted results on two-hop questions.
- Debug duplicate aliases that split one entity into three nodes.
- Add edge provenance, temporal validity, and confidence filtering.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Research Paper Graph	Links papers, authors, methods, citations	Neo4j, spaCy/LLM, FAISS	Semantic Scholar Open Research Corpus subset	Strong graph + NLP profile
Supply-chain Explorer	Answers relationship/path questions	Neo4j, FastAPI, LLM	Open supply-chain data	Enterprise graph use case
Graph-vs-Vector Study	Evaluates query classes and cost	NetworkX, SentenceTransformers	HotpotQA	Research internship value

19. Quick Revision

- **Key idea:** retrieve connected evidence, not only similar chunks.
- **Main formulas:** graph paths, bounded traversal, personalized PageRank.
- **Metrics:** entity/edge/path recall, provenance, answer faithfulness.
- **Common traps:** noisy edges, duplicate entities, uncontrolled traversal.
- **Interview one-liner:** Graph RAG pays graph-construction cost to make relationships and corpus structure first-class retrieval signals.

20. Final Cheat Sheet

Item	Summary
Input/output	Query -> ranked subgraph/paths + source passages -> answer
Main steps	extract, resolve, store, seed, traverse, rank, ground, generate
Hyperparameters	schema, confidence, depth, seed/path width, community size
Metrics	entity/edge/path recall, answer/citation quality, latency
Pros / cons	Explicit relations/global views / costly noisy maintenance
Best use cases	Relationship-heavy and multi-hop knowledge

Agentic RAG

1. Overview

Agentic RAG lets an LLM or policy dynamically decide whether, where, and how to retrieve; inspect results; call tools; reformulate; and stop. Unlike a fixed retrieve-once pipeline, it adapts the workflow to the question. It can solve heterogeneous tasks but introduces nondeterminism, loops, security risks, latency, and harder evaluation.

2. Intuition

Fixed RAG follows a recipe. Agentic RAG behaves like a researcher who decides whether to search a handbook, query a database, calculate a value, refine a search, or ask for clarification—within a controlled budget.

3. Prerequisites

- RAG components, function/tool calling, structured outputs, and state machines
- Planning, query rewriting, multi-hop reasoning, and stopping criteria
- Authentication, least privilege, idempotency, observability, and evaluation

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Router	Chooses retrieval/tool path	docs vs SQL vs no retrieval	Policy quality
Planner	Breaks goal into actions	search -> verify -> calculate	Plan vs execution
Tool contract	Typed input/output and permissions	<code>search(query, filters)</code>	Validation/security

Subtopic	Meaning and importance	Example	Interview angle
State/memory	Stores verified observations and budget	evidence IDs	Do not store guesses as facts
Reflection	Judges whether evidence is sufficient	reformulate if weak	Extra cost/correlated error
Guardrails	Restrict tools, arguments, loops, side effects	read-only search	Prompt injection defense

5. Algorithm / Working Process

1. Classify the request, risk, authorization, and whether retrieval is needed.
2. Generate a bounded plan or choose one allowed next action.
3. Validate tool name and structured arguments against trusted policy.
4. Execute the tool; treat returned content as untrusted observations.
5. Record results with provenance, not hidden unsupported conclusions.
6. Evaluate evidence sufficiency and choose answer, reformulation, another tool, clarification, or abstention.
7. Enforce hop, time, token, and monetary budgets plus repeated-state detection.
8. Construct final evidence and generate/verify a cited response.

Training may use supervised trajectories, preference data, or reinforcement learning, but many systems start with prompted tool use and deterministic orchestration.

6. Mathematical Foundation

An agent can be modeled as a policy $\pi(a_t | s_t)$ over actions given state. The objective is expected utility

$$J(\pi) = E[\text{answer_quality} - \lambda_c \cdot \text{cost} - \lambda_l \cdot \text{latency} - \lambda_r \cdot \text{risk}].$$

State transition is $s_{t+1} = T(s_t, a_t, o_t)$ after observation o_t . With stochastic actions, trajectory probability is $P(\tau) = \prod_t \pi(a_t | s_t) P(o_t | s_t, a_t)$. A constrained policy must satisfy hard authorization and budget rules regardless of expected reward.

7. Practical Implementation

```
def run_agent(question, search, max_steps=3):
    state = {"query": question, "evidence": [], "seen": set()}
    for _ in range(max_steps):
        query = state["query"]
        if query in state["seen"]: # deterministic loop guard
            break
        state["seen"].add(query)
        results = search(query) # read-only, permission-aware tool
        state["evidence"].extend(results)
        if any(r.get("answers_question") for r in results):
            return {"status": "answer", "evidence": state["evidence"]}
        state["query"] = f"More specific evidence for: {question}"
```

```
return {"status": "abstain", "evidence": state["evidence"]}  
  
demo_search = lambda q: [{"source": "S1", "answers_question": "specific" in q}]  
assert run_agent("policy?", demo_search)["status"] == "answer"
```

8. Code Explanation

This bounded loop captures the essential control plane: explicit state, a read-only tool, repeated-query detection, evidence accumulation, a success condition, and abstention after a step limit. Production systems replace the toy reformulation/sufficiency rule with validated model outputs and enforce permissions outside the model.

9. Training / Evaluation

Create task trajectories with expected tool calls, allowed/forbidden actions, evidence, and final outcomes. Evaluate task success, answer faithfulness, retrieval/tool precision/recall, invalid-call rate, loop rate, steps, latency, cost, security violations, and recovery from tool errors. Use deterministic simulations plus sandboxed integration tests. Tune route thresholds, max steps, tool descriptions, state schema, model, and budgets.

10. Complexity and Cost

Latency and cost scale with a variable number of LLM/tool calls; worst cases hit configured budgets. Parallelize independent read-only actions, cache idempotent observations, and use small routing models/rules before expensive reasoning. Tracing and replay storage add overhead but are necessary. Side-effecting tools require approvals/idempotency and are higher risk than read-only retrieval.

11. Common Use Cases

Research assistants, support diagnosis, heterogeneous enterprise search, SQL-plus-doc QA, multi-hop investigation, codebase assistants, and tasks mixing retrieval with calculators or APIs.

12. Common Mistakes

- Using an agent where one deterministic retrieval call works
- Giving the model broad tool permissions or raw credentials
- No max-step/time/cost/repeated-state guard
- Treating tool output as trusted instructions
- Keeping hidden untraceable "memory" of unsupported claims
- Evaluating only successful demos and ignoring tail cost/failure loops

13. Edge Cases / Limitations

The agent may loop, choose the wrong tool, produce malformed arguments, or stop too early. Prompt injection in retrieved text can redirect actions. Tool outages and partial failures complicate state. Nondeterministic trajectories are difficult to reproduce. Small model/prompt changes can shift routing behavior.

14. Variations

- **Corrective RAG:** assess weak retrieval and retry; practical intermediate pattern.
- **Self-RAG/adaptive retrieval:** model decides when retrieval/reflection is needed; research/interview important.
- **Router-based RAG:** deterministic/classifier routes among corpora/tools; strong production default.
- **Plan-and-execute:** separate planner and executor; useful for complex long tasks.
- **Multi-agent RAG:** specialized agents collaborate; use only when task decomposition justifies coordination cost.

15. Related Topics

Multi-hop RAG is a retrieval pattern an agent may execute. Query rewriting is one agent action. Graph RAG or SQL can be tools alongside vector search. Workflow/state-machine orchestration is more deterministic; agents are preferable only when decision paths genuinely vary.

16. Interview Questions

1. **What is Agentic RAG?** RAG where a policy dynamically chooses retrieval, tools, reformulation, and stopping actions.
2. **How differs from fixed RAG?** Fixed RAG has predetermined stages; agentic RAG selects a variable trajectory.
3. **When is it justified?** Heterogeneous sources, uncertain retrieval needs, or genuine multi-step tool use.
4. **Main risks?** Loops, injection, excessive cost, invalid actions, nondeterminism, and over-privilege.
5. **How stop loops?** Hard step/time/cost caps, repeated-state detection, and explicit terminal states.
6. **Where enforce permissions?** Deterministic tool/service layer outside the model.
7. **What belongs in state?** Goal, verified observations, provenance, unresolved needs, and budgets—not unsupported guesses.
8. **How evaluate?** Task outcome plus trajectory/tool correctness, safety, steps, latency, and cost.
9. **Reflection limitation?** The same or similar model may confidently repeat correlated errors while adding cost.
10. **Agent vs workflow?** Use deterministic workflows for known paths; agents for variable decisions that cannot be cheaply enumerated.

17. Practice Tasks

- Implement a bounded search/rewrite state machine.
- Add typed tool validation and forbidden-argument tests.
- Simulate outages, empty results, and malformed model actions.
- Red-team retrieved prompt injection attempting a tool call.
- Compare fixed, routed, and agentic pipelines on cost/success.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Docs-or-SQL Assistant	Routes questions to vector search or database	FastAPI, SQLAlchemy, vector DB, LLM	Synthetic company data	Strong tool-routing portfolio

Project	What it does	Tech stack	Dataset	Resume value
Corrective RAG Agent	Detects weak evidence and retries	LLM, reranker, eval harness	CRAG-style/custom set	Advanced RAG reliability
Agent Safety Sandbox	Tests injection, loops, budgets, permissions	Python state machine, pytest	Adversarial scenarios	High-value AI safety engineering

19. Quick Revision

- **Key idea:** controlled policy chooses retrieval/tool actions dynamically.
- **Main formula:** expected quality minus cost, latency, and risk under hard constraints.
- **Metrics:** task success, faithfulness, tool accuracy, steps, cost, violations.
- **Common traps:** unnecessary agency, no budgets, model-enforced security.
- **Interview one-liner:** Agentic RAG buys adaptive tool use at the price of a larger security, evaluation, and reliability surface.

20. Final Cheat Sheet

Item	Summary
Input/output	Goal + state/tools -> bounded trajectory -> verified answer
Main steps	route, plan/act, validate, observe, update, stop, verify
Hyperparameters	model, route threshold, max steps/time/tokens/cost
Metrics	success, faithfulness, call validity, loops, p95 cost/latency
Pros / cons	Flexible multi-tool reasoning / nondeterministic and risky
Best use cases	Variable multi-step information tasks

Fine-tuned Retrievers

1. Overview

Fine-tuned retrievers adapt a pretrained retrieval encoder to a target domain, corpus, language, or relevance definition using query-positive-negative supervision. Fine-tuning can substantially improve recall when generic embeddings misunderstand domain vocabulary or intent. Success depends heavily on label quality, negative mining, leakage control, and complete reindexing after model changes.

2. Intuition

A general librarian knows broad topics. Training on your company's real questions and correct policy passages teaches the librarian that internal acronyms, product names, and relevance rules have specialized meanings.

3. Prerequisites

- Dense retrieval, Transformer encoders, contrastive learning, and ANN

- PyTorch/Hugging Face training, batching, mixed precision, and checkpoints
- Hard-negative mining, dataset splits, retrieval evaluation, and MLOps versioning

4. Core Concepts

Subtopic	Meaning and importance	Example	Interview angle
Positive pair	Query and answer-bearing passage	support question -> policy chunk	Label quality
Negative strategy	Non-relevant comparison passages	random/BM25/hard	False negatives
In-batch negatives	Other batch passages are negatives	B-1 per query	Duplicate positives
Domain adaptation	Shift geometry to target relevance	medical/legal terms	Generalization trade-off
Distillation	Learn teacher scores/rankings	cross-encoder teacher	Soft labels
Index migration	Re-embed all documents consistently	v2 shadow index	Safe rollout

5. Algorithm / Working Process

1. Define target relevance and create a frozen, source-disjoint evaluation set.
2. Build training query-positive pairs from expert labels, logs, or carefully verified synthetic queries.
3. Add random/in-batch negatives, then mine difficult candidates with BM25/current retriever.
4. Remove known positives and likely false negatives.
5. Tokenize query and passage with correct role prefixes and train contrastively.
6. Evaluate exact retrieval before ANN to isolate encoder gains.
7. Mine new hard negatives and iterate only if validation improves.
8. Embed the full corpus into a versioned shadow index.
9. Compare quality/latency, deploy gradually, and keep rollback compatibility.

6. Mathematical Foundation

Multiple-negative ranking loss for batch size B is

$$L = -(1/B) \sum_i \log \frac{\exp(s(q_i, d_{i+}))/\tau}{\sum_j \exp(s(q_i, d_j))/\tau}.$$

With explicit negatives, denominator includes both in-batch and mined passages. A margin loss is $\max(0, m - s(q, d_+) + s(q, d_-))$. Distillation can minimize $KL(P_{\text{teacher}}(\cdot | q) || P_{\text{student}}(\cdot | q))$ over candidates. Regularization or mixing general-domain data can reduce catastrophic forgetting.

7. Practical Implementation

```
from sentence_transformers import SentenceTransformer, InputExample, losses
from torch.utils.data import DataLoader
```

```

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
examples = [
    InputExample(texts=["How many leave days carry forward?",
                        "Employees may carry forward five annual-leave days."]),
    InputExample(texts=["How do I reset my password?",
                        "Use Account Settings and select Reset Password."]),
]
loader = DataLoader(examples, batch_size=2, shuffle=True, drop_last=True)
loss = losses.MultipleNegativesRankingLoss(model)

# Practice run; use a larger held-out dataset and mined negatives in real
training.
model.fit(train_objectives=[(loader, loss)], epochs=1, warmup_steps=0)

```

8. Code Explanation

Each example contains a query and its positive passage. Within a batch, the other example's passage acts as a negative. `MultipleNegativesRankingLoss` applies the contrastive softmax objective. The tiny dataset only demonstrates the API; meaningful training requires many diverse pairs, leakage-safe evaluation, hard negatives, and checkpoint/index versioning.

9. Training / Evaluation

Split by document, organization/entity, and time; never let chunks from one source version cross train/test. Evaluate Recall@k, MRR/nDCG, per-domain/query slices, zero-shot/general-domain retention, embedding/index latency, and downstream answer quality. Tune learning rate, epochs, batch size, temperature/scale, max query/passage length, hard-negative count, model/pooling, and general-domain mixing. Stop on retrieval validation metrics, not training loss alone.

10. Complexity and Cost

Training cost is encoder forward/backward over query and passage batches plus a similarity matrix. Large batches improve negatives but require memory or cross-device gathering. Mining requires periodic corpus retrieval; re-embedding N chunks can dominate deployment cost. Serving cost may remain unchanged if the fine-tuned architecture/dimension is unchanged, but every vector must match the new model.

11. Common Use Cases

Legal/medical/scientific RAG, internal acronym-heavy knowledge, multilingual or cross-lingual retrieval, e-commerce relevance, code search, and specialized support assistants.

12. Common Mistakes

- Fine-tuning before establishing BM25/generic dense baselines
- Generating synthetic positives whose answer text leaks the query verbatim
- Using only easy random negatives or unreviewed hard negatives
- Treating alternate relevant chunks as negatives
- Selecting checkpoints by training loss instead of Recall/nDCG
- Deploying a new query encoder against old document vectors

13. Edge Cases / Limitations

Small biased datasets can overfit query templates and reduce general retrieval. Click logs encode position/presentation bias. Domain tuning may cause catastrophic forgetting. Frequently changing corpora make repeated mining/indexing expensive. If the main issue is missing data, parsing, or bad chunking, retriever fine-tuning will not solve it.

14. Variations

- **Supervised contrastive fine-tuning:** labeled pairs/negatives; highest placement relevance.
- **Weak/synthetic supervision:** LLM-generated queries or clicks; scalable but noisy.
- **Cross-encoder distillation:** transfers fine relevance signals; strong advanced project.
- **Parameter-efficient tuning:** adapters/LoRA; useful for large encoders or many domains.
- **Continual/domain-mixed training:** updates while retaining general ability; research/MLOps importance.

15. Related Topics

Dense retrieval provides the base architecture. Embedding fine-tuning reshapes its geometry. Cross-encoders supply hard labels or distillation scores. BM25 mines lexical hard negatives and remains a hybrid complement. Evaluation and index versioning are essential because a model improvement without safe migration can break production retrieval.

16. Interview Questions

1. **Why fine-tune a retriever?** To align embedding similarity with target-domain relevance and vocabulary.
2. **What data is required?** Query-positive pairs plus high-quality negatives; graded/teacher scores are optional.
3. **Why use in-batch negatives?** They provide many comparisons with few encoder passes.
4. **Why mine hard negatives?** Easy negatives quickly stop contributing useful gradients.
5. **What is a false negative?** A relevant passage labeled negative; it pushes correct evidence away.
6. **How avoid leakage?** Split by source/entity/time and remove near-duplicate/template overlap.
7. **How select checkpoints?** On held-out retrieval metrics and important slices, not training loss.
8. **Why reindex?** Updated document encoder geometry makes old vectors incompatible.
9. **How use a cross-encoder teacher?** Score/rank candidates and distill its distribution or margins into the bi-encoder.
10. **When should you not fine-tune?** When failures come from corpus coverage, parsing, chunking, filters, or insufficient evaluation data.

17. Practice Tasks

- Fine-tune with MultipleNegativesRankingLoss and evaluate Recall@10.
- Compare random, BM25, and dense hard-negative mining.
- Find false negatives by pooling judgments from multiple retrievers.
- Measure domain gains versus general BEIR regression.
- Design a shadow-index rollout with model/vector version checks.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Scientific Retriever	Adapts embeddings to scientific claims	SentenceTransformers, FAISS	SciFact	Strong domain NLP project
Teacher-Student Retriever	Distills cross-encoder relevance	PyTorch, HF	MS MARCO/BEIR	Research-depth ranking work
Safe Retriever Migration	Builds shadow index and regression gates	MLflow, Qdrant, FastAPI	Custom enterprise corpus	Excellent production/MLOps signal

19. Quick Revision

- **Key idea:** learn target relevance geometry from positives and difficult negatives.
- **Main formula:** contrastive softmax; optional teacher KL/margin loss.
- **Metrics:** Recall@k, MRR/nDCG, domain slices, general retention.
- **Common traps:** false negatives, leakage, and query/index version mismatch.
- **Interview one-liner:** fine-tuning pays off when relevance—not corpus coverage or parsing—is the measured bottleneck.

20. Final Cheat Sheet

Item	Summary
Input/output	Labeled query-passage data -> adapted encoders and new index
Main steps	label, split, mine negatives, train, evaluate, re-embed, shadow deploy
Hyperparameters	model, LR, epochs, batch, temperature, negatives, lengths
Metrics	Recall@k, MRR/nDCG, general retention, latency
Pros / cons	Domain relevance gains / data, training, and reindexing cost
Best use cases	Stable specialized domains with measurable generic-retriever gaps